Data Communications & Networks

Chapter 3: Sections 3.5 & 3.6

Data Rate Limits & Network Performance

Topics Covered:

- Section 3.5 Data Rate Limits (Nyquist & Shannon)
- Section 3.6 Performance: Bandwidth, Throughput, Latency, Bandwidth-Delay Product, Jitter

Reference: Forouzan, "Data Communications and Networking," 5th ed.

April 11, 2026

Contents

1 Data Rate Limits (Section 3.5) 2

1.1 The Nyquist Bit Rate Formula (Noiseless Channel) . . . 2

1.1.1 Background and Intuition . . . 2

1.1.2 The Nyquist Formula . . . 3

1.1.3 Worked Examples . . . 3

1.2 Signal-to-Noise Ratio (SNR) — A Bridge to Shannon . . . 4

1.2.1 What is SNR? . . . 4

1.3 Shannon's Capacity Formula (Noisy Channel) . . . 5

1.3.1 The Shannon Formula . . . 5

1.3.2 Key Insight: Shannon Vs. Nyquist . . . 6

1.3.3 Worked Examples with Shannon . . . 6

1.3.4 Quick Reference Table: Nyquist vs Shannon . . . 7

2 Performance (Section 3.6) 8

2.1 Bandwidth . . . 8

2.1.1 Two Meanings of Bandwidth . . . 8

2.2 Throughput . . . 9

2.2.1 Definition and Distinction from Bandwidth . . . 9

2.3 Latency (Delay) . . . 9

2.3.1 Definition . . . 10

2.3.2 Component 1: Propagation Time . . . 10

2.3.3 Component 2: Transmission Time . . . 10

2.3.4 Component 3: Queuing Time . . . 11

2.3.5 Component 4: Processing Time . . . 12

2.3.6 Total Latency Formula . . . 12

2.4 Bandwidth-Delay Product . . . 13

2.4.1 Concept and Intuition . . . 13

2.5 Jitter . . . 14

2.5.1 What is Jitter? . . . 14

2.5.2 Why Does Jitter Occur? . . . 14

2.5.3 Handling Jitter: The Playout Buffer . . . 15

2.6 Summary Table: All Performance Metrics . . . 15

Chapter 1 Data Rate Limits (Section 3.5)

What is a Data Rate Limit?

Imagine you are pouring water from a bucket through a pipe into another bucket. The maximum speed at which you can transfer water depends on: (1) how wide the pipe is, (2) how many lanes the pipe has, and (3) how clean the water is (free from impurities/noise).
In data communications, **data rate** is how fast we can send bits per second (bps). The maximum data rate depends on:

1. The **bandwidth** of the channel (Hz)
2. The **number of signal levels** used
3. The **quality of the channel** (noise level)

Two mathematical formulas — one by **Nyquist** and one by **Shannon** — let us calculate the theoretical maximum data rate. Knowing this maximum is crucial because it tells you the ceiling you can never exceed, no matter how clever your technology.

1.1 The Nyquist Bit Rate Formula (Noiseless Channel)

1.1.1 Background and Intuition

Before understanding Nyquist, let's understand what a **signal level** means.

A signal level is just a distinct voltage (or amplitude) we assign a meaning to. For example:

- **2 levels**: 0 Volts = binary 0, +5 Volts = binary 1. Each level carries 1 bit.
- **4 levels**: 00, 01, 10, 11 can be represented by four voltages. Each level carries 2 bits.
- **8 levels**: Each level represents 3 bits ($2^3 = 8$).
- **L levels**: Each level represents $\log_2 L$ bits.

So the more levels we use, the more data we carry per signal change. But there is a catch: the receiver becomes more confused when there are too many similar levels, especially when noise is present.

1.1.2 The Nyquist Formula

Nyquist Bit Rate Formula

For a **noiseless** channel:

$$\boxed{\text{BitRate} = 2 \times \text{Bandwidth} \times \log_2 L}$$

Where:

- **BitRate** = maximum bits per second (bps)
- **Bandwidth** = bandwidth of the channel in Hertz (Hz)
- **L** = number of signal levels used
- $\log_2 L$ = number of bits each signal level represents

Why is there a factor of 2? Nyquist proved that for a bandwidth B Hz, the maximum number of signal changes (baud rate) possible without inter-symbol interference is $2B$ per second. Each signal change carries $\log_2 L$ bits, giving the formula above.

1.1.3 Worked Examples

Example 3.33 (Nyquist): Two signal levels, 3000 Hz bandwidth

Question: Consider a noiseless channel with a bandwidth of 3000 Hz transmitting a signal with two signal levels. What is the maximum bit rate?

Solution:

$$\text{BitRate} = 2 \times 3000 \times \log_2 2 = 2 \times 3000 \times 1 = \mathbf{6000} \text{ bps}$$

With only 2 levels, each signal change carries only 1 bit. So we get 6000 bps maximum.

Example: Four signal levels, 3000 Hz bandwidth

Question: Same channel, but now using 4 signal levels. What is the maximum bit rate?

Solution:

$$\text{BitRate} = 2 \times 3000 \times \log_2 4 = 2 \times 3000 \times 2 = \mathbf{12000} \text{ bps}$$

By doubling the number of signal levels (from 2 to 4), we doubled the bit rate!

Example: Eight signal levels, 3000 Hz bandwidth

Question: Same channel, now using 8 signal levels.

Solution:

$$\text{BitRate} = 2 \times 3000 \times \log_2 8 = 2 \times 3000 \times 3 = \mathbf{18000} \text{ bps}$$

Pattern observed: As L increases, bit rate increases. But this has a practical limit!

Reverse Problem: Finding L from a required bit rate

Question: We have a noiseless channel with 1 MHz bandwidth. We want a data rate of 4 Mbps. What is the minimum number of signal levels needed?

Solution:

$$\begin{aligned} 4{,}000{,}000 &= 2 \times 1{,}000{,}000 \times \log_2 L \\ \log_2 L &= \frac{4{,}000{,}000}{2{,}000{,}000} = 2 \\ L &= 2^2 = \mathbf{4} \text{ signal levels} \end{aligned}$$

Important Insight: Increasing L increases bit rate mathematically, but practically we cannot increase L indefinitely. When there are too many levels, even tiny noise makes the receiver confuse one level with another. So Nyquist gives the upper bound only for a noiseless channel. In reality, noise exists, which is where Shannon's formula becomes essential.

1.2 Signal-to-Noise Ratio (SNR) — A Bridge to Shannon

Before we can understand Shannon's formula, we need to understand SNR.

1.2.1 What is SNR?

Signal-to-Noise Ratio

Noise is unwanted electrical energy that gets added to your signal during transmission (from motors, other cables, weather, etc.). SNR measures how "strong" the useful signal is compared to the noise.

$$\text{SNR} = \frac{\text{Average Signal Power}}{\text{Average Noise Power}}$$

SNR expressed in decibels (dB):

$$\text{SNR}_{dB} = 10 \log_{10}(\text{SNR})$$

- **High SNR** ⇒ Signal is much stronger than noise ⇒ easy to distinguish levels ⇒ higher data rate possible.
- **Low SNR** ⇒ Noise dominates ⇒ hard to distinguish levels ⇒ lower data rate.
- **Ideal (no noise)** ⇒ SNR = ∞.

Example 3.31: Calculate SNR and SNR_{dB}

Given: Signal power = 10 mW, Noise power = 1 μW = 0.001 mW

Solution:

$$\text{SNR} = \frac{10 \text{ mW}}{0.001 \text{ mW}} = \frac{10,000}{1} = \mathbf{10,000}$$

$$\text{SNR}_{dB} = 10 \log_{10}(10,000) = 10 \log_{10}(10^4) = 10 \times 4 = \mathbf{40} \text{ dB}$$

Meaning: The signal is 10,000 times stronger than the noise. This is a very clean channel!

Example 3.32: Perfect noiseless channel

Given: No noise at all (noise power = 0).

Solution:

$$\text{SNR} = \frac{\text{Signal Power}}{0} = \infty \quad \Rightarrow \quad \text{SNR}_{dB} = 10 \log_{10}(\infty) = \infty$$

Note: This is theoretical only. In practice, SNR is always finite.

1.3 Shannon's Capacity Formula (Noisy Channel)

1.3.1 The Shannon Formula

Shannon's Channel Capacity Formula

For a **noisy** channel:

$$\boxed{C = B \times \log_2(1 + \text{SNR})}$$

Where:

- C = Channel capacity (maximum data rate in bps)
- B = Bandwidth of the channel in Hz
- SNR = Signal-to-Noise Ratio (not in dB, just the plain ratio)
- $\log_2(1 + \text{SNR})$ = theoretical maximum bits per signal cycle

Note: Shannon's formula gives an *absolute upper bound* that cannot be exceeded regardless of the number of signal levels. It accounts for noise, unlike Nyquist.

1.3.2 Key Insight: Shannon Vs. Nyquist

Nyquist Formula
BitRate $= 2 \times B \times \log_2 L$
Assumes: No noise

Combined gives

Shannon Formula
$C = B \times \log_2(1 + \mathbf{SNR})$
Assumes: Noise present

- **Shannon** tells you the *theoretical ceiling* (capacity). You can never go faster than this.
- **Nyquist** tells you how many signal levels L you need to hit that ceiling.
- In practice, you use **both**: Shannon gives you C, then Nyquist lets you find L.

1.3.3 Worked Examples with Shannon

Example 3.34: Telephone line capacity

Given: Telephone line bandwidth = 3000 Hz. Typical noise makes SNR $\approx$ 3162.

Solution:

$$C = 3000 \times \log_2(1 + 3162) = 3000 \times \log_2(3163)$$

$\log_2(3163) = \frac{\ln(3163)}{\ln 2} = \frac{8.059}{0.693} \approx 11.63$

$$C = 3000 \times 11.63 \approx \mathbf{34,860} \text{ bps} \approx \mathbf{35} \text{ kbps}$$

Meaning: No matter what tricks we use (compression, clever modulation), a phone line with this SNR can NEVER carry more than about 35 kbps. This is why traditional dial-up modems maxed out around 33.6 kbps!

Example 3.35: Extremely noisy channel

Question: The value of the SNR is almost zero (noise overwhelms the signal). Find capacity.

Solution:

$$C = B \times \log_2(1 + 0) = B \times \log_2(1) = B \times 0 = \mathbf{0} \text{ bps}$$

Meaning: If the noise drowns out the signal completely, no data transfer is possible, regardless of bandwidth. A channel buried in noise is useless!

Example 3.36: Finding number of levels using both formulas

Given: A channel has bandwidth = 1 MHz and SNR = 63. Find: (a) Theoretical maximum data rate using Shannon. (b) Number of signal levels L needed using Nyquist.

Solution (a) — Shannon:

$$C = 1{,}000{,}000 \times \log_2(1 + 63) = 1{,}000{,}000 \times \log_2(64) = 1{,}000{,}000 \times 6 = \mathbf{6} \text{ Mbps}$$

Solution (b) — Nyquist to find L:

We want the bit rate to match Shannon's capacity of 6 Mbps:

$$\begin{aligned} 6{,}000{,}000 &= 2 \times 1{,}000{,}000 \times \log_2 L \\ \log_2 L &= 3 \\ L &= 2^3 = \mathbf{8} \text{ signal levels} \end{aligned}$$

Conclusion: We need 8 distinct signal levels to achieve the maximum data rate of 6 Mbps on this channel.

Example 3.37: Combining Nyquist & Shannon for 4 Mbps at 1 MHz

Given: Channel bandwidth = 1 MHz. We need 4 Mbps. Shannon gives us 6 Mbps ceiling. So 4 Mbps is achievable! Now find L using Nyquist.

Solution:

$$\begin{aligned} 4{,}000{,}000 &= 2 \times 1{,}000{,}000 \times \log_2 L \\ \log_2 L &= 2 \\ L &= 4 \text{ signal levels} \end{aligned}$$

The Shannon capacity gives us the upper limit; the Nyquist formula tells us how many signal levels we need.

1.3.4 Quick Reference Table: Nyquist vs Shannon

Feature	**Nyquist**	**Shannon**
Channel type	Noiseless	Noisy
Formula	$2 \times B \times \log_2 L$	$B \times \log_2(1 + \text{SNR})$
Parameter	Number of levels L	SNR ratio
Purpose	Find max data rate	Find absolute capacity
Use case	Find L needed	Find theoretical ceiling

Chapter 2 Performance (Section 3.6)

What is Performance?

Performance in networking is like judging a highway system. You ask:

- How *wide* is the highway? (Bandwidth)
- How many cars actually pass per hour? (Throughput)
- How long does a car take from start to finish? (Latency)
- Can the highway hold many cars at once? (Bandwidth-Delay Product)
- Are cars arriving at different times? (Jitter)

The same questions apply to data networks. This section introduces five key performance metrics.

2.1 Bandwidth

2.1.1 Two Meanings of Bandwidth

The word "bandwidth" is used in two very different but related ways in networking:

Context	Unit	Meaning
In Hertz (Hz)	Hertz	Range of frequencies a medium can carry
In bps	bits per second	Maximum amount of data a link can carry

Bandwidth in Hz

This is the *physical* bandwidth. It defines how wide a range of frequencies the transmission medium supports. For example, a telephone line supports frequencies from 300 Hz to 3400 Hz, so its bandwidth in Hz is $3400 - 300 = 3100$ Hz. The more Hz of bandwidth, the higher the potential data rate (see Nyquist & Shannon).

Bandwidth in bps

This is the *network-layer* bandwidth. It is the maximum number of bits that can be sent per second on the link. This is what engineers usually mean in everyday conversation: "We have a 100 Mbps link."

Example 3.41: Telephone voice bandwidth

A subscriber line has a bandwidth of 4 kHz for voice. For data transmission (using a modem), this bandwidth can support up to **56,000 bps**. The telephone company limits voice calls to 4 kHz, but a 56K modem uses sophisticated signal encoding to push data at 56 kbps.

Example 3.42: DSL bandwidth

If the telephone company can replace the 4-kHz filter with a 1-MHz filter, the home subscriber line can carry far more data. DSL (Digital Subscriber Line) uses wider bandwidths for this reason.

2.2 Throughput

2.2.1 Definition and Distinction from Bandwidth

Throughput

Throughput is the *actual* data transfer rate measured in practice, as opposed to the *theoretical* maximum called bandwidth.

$$\text{Throughput} \leq \text{Bandwidth}$$

Throughput is always less than or equal to bandwidth because:

- Network congestion reduces available capacity
- Protocol overhead (headers, acknowledgments) consumes bandwidth
- Hardware limitations slow things down
- Errors cause retransmissions

Example: Bandwidth vs Throughput

You have a 1 Gbps Ethernet link at home. You download a file and measure 400 Mbps actual download speed. Therefore:

- Bandwidth = **1 Gbps** (theoretical maximum)
- Throughput = **400 Mbps** (what you actually got)

The gap exists because of congestion on the internet path, TCP protocol overhead, etc.

2.3 Latency (Delay)

2.3.1 Definition

Latency

Latency (also called **delay**) is the total time it takes for a message to travel from sender to receiver. It is made of four components:

$$\text{Latency} = \text{Propagation Time} + \text{Transmission Time} + \text{Queuing Time} + \text{Processing Time}$$

2.3.2 Component 1: Propagation Time

Propagation Time

This is the time for a signal (electromagnetic wave or light pulse) to physically travel through the transmission medium from point A to point B.

$$\text{Propagation Time} = \frac{\text{Distance}}{\text{Propagation Speed}}$$

Propagation speed depends on the medium:

- Vacuum / Free air: 3×10^8 m/s (speed of light)
- Copper wire: typically 2×10^8 m/s
- Optical fiber: typically 2×10^8 m/s

Example 3.45: Propagation time across a link

Question: What is the propagation time if the distance between two points is 12,000 km and the signal propagates through a medium at 2.4×10^8 m/s?

Solution:

$$\text{Propagation Time} = \frac{12{,}000{,}000 \text{ m}}{2.4 \times 10^8 \text{ m/s}} = \frac{1.2 \times 10^7}{2.4 \times 10^8} = 0.05 \text{ s} = \mathbf{50} \text{ ms}$$

It takes 50 ms for a signal to travel 12,000 km. This is why you notice a slight delay when calling long-distance internationally — the signal itself has to travel thousands of kilometers.

2.3.3 Component 2: Transmission Time

Transmission Time

This is the time it takes to put all the bits of a message onto the transmission medium (like loading all passengers onto a train before it departs).

$$\text{Transmission Time} = \frac{\text{Message Size (bits)}}{\text{Bandwidth (bps)}}$$

Example: Transmission time for a large file

Question: How long does it take to transmit 2.5 GB of data on a 1 Gbps link?

Solution:

$$\text{Message Size} = 2.5 \times 10^9 \times 8 = 20 \times 10^9 \text{ bits} = 20 \text{ Gbits}$$
$$\text{Transmission Time} = \frac{20 \times 10^9}{10^9} = \mathbf{20} \text{ seconds}$$

Even on a 1 Gbps link, a 2.5 GB file takes 20 seconds just to be put on the wire!

Example 3.47: Dominant factor comparison

Given: A message of 100 MB ($= 800 \times 10^6$ bits) is sent on a 1 Mbps link over a distance of 1000 km (propagation speed $= 2 \times 10^8$ m/s). Find total latency.

Solution:

$$\text{Transmission Time} = \frac{800 \times 10^6}{1 \times 10^6} = \mathbf{800} \text{ s}$$
$$\text{Propagation Time} = \frac{1{,}000{,}000}{2 \times 10^8} = 0.005 \text{ s} = \mathbf{5} \text{ ms}$$

Total Latency $\approx$ 800 seconds (propagation is negligible here).
Dominant factor: **Transmission time** (because the message is huge and the bandwidth is small).

Example 3.48: Propagation dominates for small messages on fast links

Given: A 1-bit message (single bit!) is sent over a 1 Gbps link spanning 1000 km.

Solution:

$$\text{Transmission Time} = \frac{1}{10^9} = 1 \text{ ns (nanosecond)}$$
$$\text{Propagation Time} = \frac{10^6}{2 \times 10^8} = 5 \text{ ms}$$

Total Latency $\approx$ 5 ms (transmission time is negligible).
Dominant factor: **Propagation time** (because the message is tiny but the distance is large).

2.3.4 Component 3: Queuing Time

Queuing Time

When data reaches an intermediate device (router or switch), it may have to *wait in a queue* if the device is busy processing earlier data. This waiting time is **queuing time**.

Queuing time is not fixed — it varies based on network load. During peak hours, queuing time increases significantly.

2.3.5 Component 4: Processing Time

When a router receives a data packet, it must:

1. Read the destination address.
2. Look up the routing table.
3. Decide which output port to use.

This takes some time, called **processing time**.

2.3.6 Total Latency Formula

Total Latency Calculation

$$\text{Latency} = t_{\text{prop}} + t_{\text{trans}} + t_{\text{queue}} + t_{\text{proc}}$$

$$t_{\text{prop}} = \frac{d}{s}, \quad t_{\text{trans}} = \frac{n}{B}$$

Where: d = distance, s = propagation speed, n = message size (bits), B = bandwidth (bps)

Example P3.33 (Comprehensive Latency Problem)

Given:

- Frame size: 5,000,000 bits
- 10 routers, each with: queuing time = 2 μs, processing time = 1 μs
- Link length: 2000 km, propagation speed = 2×10^8 m/s
- Bandwidth: 10 Mbps

Find: Total latency.

Solution:

$$t_{\text{trans}} = \frac{5{,}000{,}000}{10{,}000{,}000} = 0.5 \text{ s}$$

$$t_{\text{prop}} = \frac{2{,}000{,}000}{2 \times 10^8} = 0.01 \text{ s} = 10 \text{ ms}$$

$$t_{\text{queue}} = 10 \times 2 \times 10^{-6} = 20 \times 10^{-6} \text{ s} = 20\ \mu\text{s}$$

$$t_{\text{proc}} = 10 \times 1 \times 10^{-6} = 10 \times 10^{-6} \text{ s} = 10\ \mu\text{s}$$

$$\begin{aligned}\text{Total} &= 0.5 + 0.01 + 0.000020 + 0.000010 \\ &\approx \mathbf{0.51003} \text{ s} \approx \mathbf{510} \text{ ms}\end{aligned}$$

The dominant factor is transmission time (0.5 s). Queuing and processing are negligible.

2.4 Bandwidth-Delay Product

2.4.1 Concept and Intuition

Bandwidth-Delay Product (BDP)

The **Bandwidth-Delay Product** tells us the *maximum amount of data that can be "in flight" on a link* at any given moment — data that has been sent but not yet received.

$$\text{BDP} = \text{Bandwidth} \times \text{Delay (Propagation time)}$$

Analogy: Think of the link as a pipe. Bandwidth is the diameter of the pipe, and propagation delay is the length of the pipe. The BDP is the *volume* of the pipe — how much water (data) it can hold at once.

- **High BDP** = "fat long pipe" = many bits can be in transit simultaneously.
- **Low BDP** = "thin short pipe" = few bits in transit.

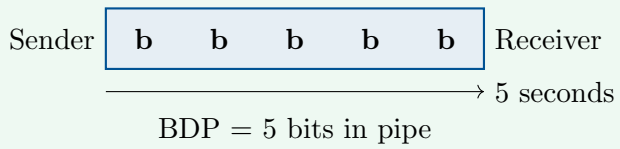
This matters for designing efficient protocols. If BDP is large and the sender stops to wait for an ACK after every packet, most of the pipe is wasted!

Example 3.49 (Case 1): Narrow/Short Pipe

Given: Bandwidth = 1 bps, Propagation delay = 5 s.

BDP $= 1 \times 5 =$ **5** bits

This means at any moment, at most 5 bits can be "on the wire" simultaneously.

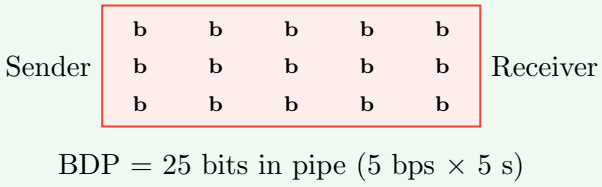


Example 3.50 (Case 2): Wide/Long Pipe (High BDP)

Given: Bandwidth = 5 bps, Propagation delay = 5 s.

BDP $= 5 \times 5 =$ **25** bits

At any moment, up to 25 bits are in transit simultaneously.



Real-World BDP Example: Intercontinental Fiber

Given: 1 Gbps link, propagation delay = 50 ms (typical trans-Atlantic cable).

BDP:

$$\text{BDP} = 10^9 \text{ bps} \times 0.05 \text{ s} = 50{,}000{,}000 \text{ bits} = \mathbf{50} \text{ Mb} = \mathbf{6.25} \text{ MB}$$

At any moment, 6.25 MB of data can be "in flight" across the Atlantic! This is why protocols like TCP need large window sizes for efficient use of high-BDP links.

2.5 Jitter

2.5.1 What is Jitter?

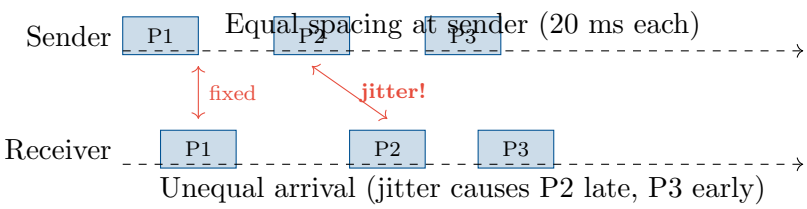
Jitter — The Variability of Delay

Jitter is the *variation in the delay* experienced by successive packets.

- If a video stream sends packets every 20 ms, the receiver expects them every 20 ms.
- If one packet arrives after 19 ms, the next after 25 ms, the next after 21 ms — the delivery times are varying. This variation is jitter.
- High jitter causes choppy audio, pixelated video, and poor real-time communication.

$$\text{Jitter} = \text{Variation in Delay Between Packets}$$

2.5.2 Why Does Jitter Occur?



Jitter occurs because:

- Different packets may take different paths through the network.
- Router queuing times vary based on traffic load.
- Packets may need retransmission due to errors.

2.5.3 Handling Jitter: The Playout Buffer

Real-time applications (voice, video) handle jitter using a **playout buffer**:

1. Incoming packets are stored in a buffer.
2. Packets are played out at a fixed, regular rate.
3. The buffer absorbs the variation (jitter) in arrival times.
4. If the buffer is too small: gaps occur when packets arrive late.
5. If the buffer is too large: extra latency is introduced.

2.6 Summary Table: All Performance Metrics

Metric	Formula	Measures
Bandwidth (Hz)	—	Range of supported frequencies
Bandwidth (bps)	—	Maximum data rate on link
Throughput	Measured value $\leq$ BW	Actual data rate achieved
Propagation time	d/s	Signal travel time through medium
Transmission time	n/B	Time to put all bits on wire
Total Latency	$t_{\text{prop}} + t_{\text{trans}} + t_{\text{queue}} + t_{\text{proc}}$	Total end-to-end delay
Bandwidth-Delay Product	BW $\times$ Propagation delay	Max bits "in flight" simultaneously
Jitter	Variation in delay	Smoothness of packet delivery

Chapter Summary

Key Takeaways

Section 3.5 — Data Rate Limits:

- **Nyquist**: For a noiseless channel, BitRate $= 2B \log_2 L$. More signal levels = higher rate, but less reliability.
- **Shannon**: For a noisy channel, $C = B \log_2(1 + \text{SNR})$. This is the absolute ceiling; you can never exceed it.
- Use **Shannon** to find the capacity ceiling, then **Nyquist** to find how many levels L are needed.
- $\text{SNR}_{dB} = 10 \log_{10}(\text{SNR})$.

Section 3.6 — Performance:

- **Bandwidth** = maximum rate (theoretical). **Throughput** = actual rate (always $\leq$ bandwidth).
- **Latency** = Propagation + Transmission + Queuing + Processing time.
- **Propagation** $= d/s$ dominates for small messages over long distances.
- **Transmission** $= n/B$ dominates for large messages over short-distance fast links.
- **BDP** = Bandwidth $\times$ Delay = how many bits can be "in the pipe" at once.
- **Jitter** = variation in packet delay; critical for real-time applications like VoIP/video.

Data Communications & Networks

Chapter 4: Sections 4.1.1 & 4.3

Line Coding (Theory) & Transmission Modes

Topics Covered:

- Section 4.1.1 Line Coding: Theory Only
- Section 4.3 Transmission Modes: Parallel & Serial (Asynchronous, Synchronous)

Reference: Forouzan, "Data Communications and Networking," 5th ed.

April 11, 2026

Contents

1 Line Coding — Section 4.1.1 (Theory) 2

1.1 Characteristics of Line Coding . . . 2

1.1.1 Signal Element vs. Data Element . . . 2

1.1.2 Data Rate vs. Signal Rate . . . 3

1.1.3 Baseline Wandering . . . 3

1.1.4 DC Component . . . 4

1.1.5 Self-Synchronization . . . 4

1.2 Classification of Line Coding Schemes . . . 4

1.3 Unipolar Schemes . . . 5

1.4 Polar Schemes . . . 5

1.4.1 Polar NRZ: NRZ-L and NRZ-I . . . 6

1.4.2 Return-to-Zero (RZ) . . . 7

1.4.3 Biphase: Manchester and Differential Manchester . . . 7

1.5 Bipolar Schemes . . . 8

1.6 Multilevel Schemes (Brief Theory) . . . 9

1.7 Key Comparison Table . . . 9

2 Transmission Modes — Section 4.3 10

2.1 Parallel Transmission . . . 10

2.2 Serial Transmission . . . 11

2.2.1 Asynchronous Serial Transmission . . . 11

2.2.2 Synchronous Serial Transmission . . . 12

2.3 Comparison: Parallel vs. Serial vs. Async vs. Sync . . . 13

Chapter 1 Line Coding — Section 4.1.1 (Theory)

What is Line Coding?

Computers work with digital data — a stream of 0s and 1s. To send this data across a wire or fiber optic cable, we must convert these 0s and 1s into a **digital signal** (a stream of electrical voltage pulses). The process of converting digital data (bits) into a digital signal is called **line coding**.
Think of it this way: data is what you want to say; the signal is how you say it over the wire. Different line coding schemes say the same thing (same bits) in different "accents."
Why do we need different schemes? Because different situations demand different properties: some schemes need perfect synchronization, others save bandwidth, some handle DC components better. Line coding is the engineer's toolkit for this.

1.1 Characteristics of Line Coding

Before examining specific schemes, we need to understand the key characteristics that any line coding scheme is measured against:

1.1.1 Signal Element vs. Data Element

Signal Element and Data Element

- A **data element** is the smallest piece of information: a single **bit**.
- A **signal element** is the shortest unit of a digital signal on the wire.
- The ratio $r = \frac{\text{data elements}}{\text{signal elements}}$ tells us the efficiency.
- $r > 1$: Each signal element carries more than 1 bit (efficient!).
- $r = 1$: One signal element per bit.
- $r < 1$: Need more than one signal element per bit (less efficient, but has advantages).

1.1.2 Data Rate vs. Signal Rate

Data Rate (Bit Rate) vs Signal Rate (Baud Rate)

- **Data rate** = number of bits per second (bps) = *what we want*
- **Signal rate** (baud rate) = number of signal elements per second = *what the wire sees*
- Relationship: Signal Rate = $\frac{\text{Data Rate}}{r}$, where $r = \log_2 L$ for L levels.
- We want **high data rate** with **low signal rate** (uses less bandwidth).

The Nyquist formula gives us the maximum data rate for a given signal rate:

$$N_{max} = \frac{1}{c} \times B \times r$$

Where c is a constant that depends on the line coding scheme (worst case $c = 1$, best case $c = 0.5$, average $c \approx 1$), B is bandwidth, and $r = \log_2 L$.

Example 4.1: Understanding data vs signal rate

For a binary signal (2 levels), $r = 1$, so if the baud rate is 1000 baud, the data rate is 1000 bps.
For a 4-level signal, $r = \log_2 4 = 2$, so 1000 baud carries 2000 bps. Same bandwidth, double the data!

Example 4.2: Verifying with Nyquist

The Nyquist formula: $N_{max} = 2 \times B \times \log_2 L$ (from Chapter 3).
For $c = 1$ (worst case): $N = \frac{1}{1} \times B \times r = B \times \log_2 L$. For $c = \frac{1}{2}$ (best case): $N = \frac{1}{0.5} \times B \times r = 2B \times \log_2 L$. This matches Nyquist!
So line coding operating at best case ($c = 0.5$) achieves the Nyquist limit.

1.1.3 Baseline Wandering

Baseline Wandering

The **baseline** is the running average of the received signal power. The receiver uses this baseline to decode signal levels. If the signal has a long string of 1s or 0s, the baseline "wanders" (drifts) from the correct value. When baseline wanders, the receiver makes more errors.
Good line coding schemes avoid long runs of the same signal level to prevent baseline wandering.

1.1.4 DC Component

DC Component

A signal with a **DC component** has a non-zero average value. For example, if 1 is represented by +5V and 0 by 0V, then a stream of all-1s has a DC component.
Why is DC bad?

- Some transmission media (e.g., telephone lines using transformers) cannot pass DC.
- DC wastes energy without carrying information.
- Causes baseline wandering.

Good line coding schemes are **DC-balanced** (equal positive and negative voltages over time, average = 0).

1.1.5 Self-Synchronization

Self-Synchronization

For the receiver to correctly read bits, it must know *exactly when* each bit starts and ends. This requires **synchronization** between sender and receiver clocks.
A **self-synchronizing** signal contains timing information within the signal itself. When the signal changes level (a transition), the receiver can use that transition to re-synchronize its clock. No separate clock signal is needed.
Problem without synchronization: If sender and receiver clocks drift apart by even 0.1%, over 1 million bits the receiver will be off by 1000 bits — reading incorrect data!

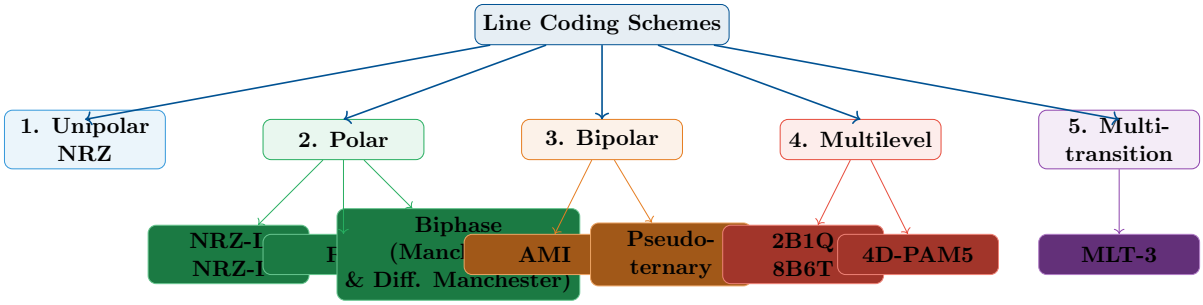
Example 4.3: Synchronization error

Suppose a sender transmits at 1 Mbps and the receiver runs at 1.001 Mbps (0.1% drift). After sending 1,000,000 bits, the receiver has "read" 1,001,000 bits — it has invented 1000 extra bits that were never sent! The entire data stream becomes corrupted. This is why self-synchronization is critical.

1.2 Classification of Line Coding Schemes

The Line Coding Family Tree

Line coding schemes are organized into five major categories:



1.3 Unipolar Schemes

Unipolar NRZ (Non-Return-to-Zero)

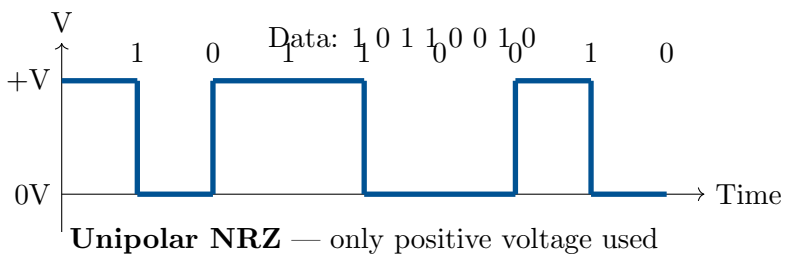
Unipolar means signals use *only one polarity*: all positive (or all negative).

- **Binary 1** → Positive voltage (+V)
- **Binary 0** → Zero volts (0V)
- **NRZ** = Non-Return-to-Zero: the signal does NOT return to 0 between consecutive 1s.

Problems:

- **DC component**: A stream of 1s = constant +V (pure DC). Transformers can't pass it.
- **No self-synchronization**: A long run of 1s with no transitions = receiver can't sync.
- **Baseline wandering**: Average voltage shifts with data.

Use: Mainly in labs; not used in real networks due to these problems.



Unipolar NRZ — only positive voltage used

1.4 Polar Schemes

Polar schemes use both **positive** and **negative** voltages, eliminating the DC problem (when balanced). Four main polar schemes are used:

1.4.1 Polar NRZ: NRZ-L and NRZ-I

NRZ-L (Non-Return-to-Zero Level)

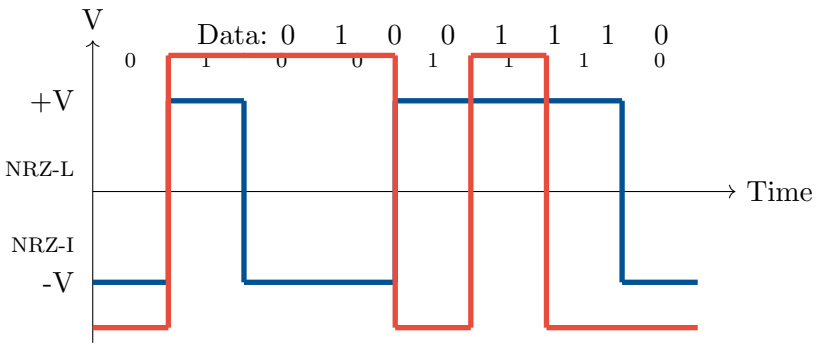
- **Binary 1** → Positive voltage (+V)
- **Binary 0** → Negative voltage (-V)
- Level determines bit: hence "L" for Level.

Advantage over unipolar: No DC component if data is balanced (equal 1s and 0s).
Problem: Long runs of same bit = no transitions = loss of synchronization.

NRZ-I (Non-Return-to-Zero Invert)

- **Binary 1** → Change (invert) the voltage level.
- **Binary 0** → No change; stay at current level.
- The transition itself encodes the bit, not the level.

Advantage: Long strings of 1s produce transitions → better synchronization for 1s.
Problem: Long strings of 0s = no transitions = synchronization problem for 0s.
Used in: USB (with NRZI-type encoding).



Blue = NRZ-L, Red = NRZ-I

Example 4.4: NRZ-I Bandwidth Calculation

Given: System uses NRZ-I to transfer data at 10 Mbps. Find average signal rate and minimum bandwidth.

Solution: For NRZ-I, $r = 1$ (one bit per signal element), $c = \frac{1}{2}$ (average case):

$$S_{avg} = \frac{N}{2} = \frac{10 \times 10^6}{2} = \mathbf{5 \times 10^6} \text{ baud} = \mathbf{5} \text{ Mbaud}$$

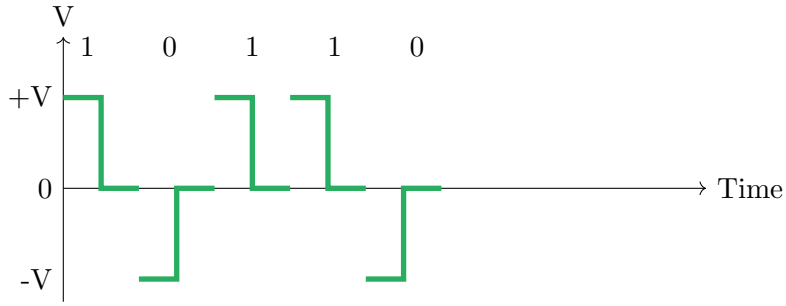
Minimum bandwidth = S_{avg} = **5** MHz.
Why divide by 2? For random data, transitions occur on average half the time.

1.4.2 Return-to-Zero (RZ)

RZ Scheme

- **Binary 1** → Start at +V, return to 0V in the middle of the bit period.
- **Binary 0** → Start at -V, return to 0V in the middle of the bit period.

Advantage: The transition back to zero in the middle of EVERY bit provides excellent self-synchronization. Even a long run of 1s has a transition at mid-bit.
Disadvantage: Requires **twice** the signal rate of NRZ (needs twice the bandwidth).
Two signal changes per bit period instead of one.



RZ — transitions at mid-bit ensure synchronization

1.4.3 Biphase: Manchester and Differential Manchester

Manchester Encoding

Manchester encodes each bit with a **transition in the middle** of the bit period:

- **Binary 1** → Transition from **low to high** (negative to positive) at mid-bit.
- **Binary 0** → Transition from **high to low** (positive to negative) at mid-bit.

Advantages:

- **Excellent synchronization**: Every bit has a mid-bit transition.
- **No DC component**: Each bit contributes equal positive and negative time.
- No baseline wandering.

Disadvantage: Signal rate is **double** the bit rate (needs twice the bandwidth).
Used in: Ethernet (10BASE-T), older LAN standards.

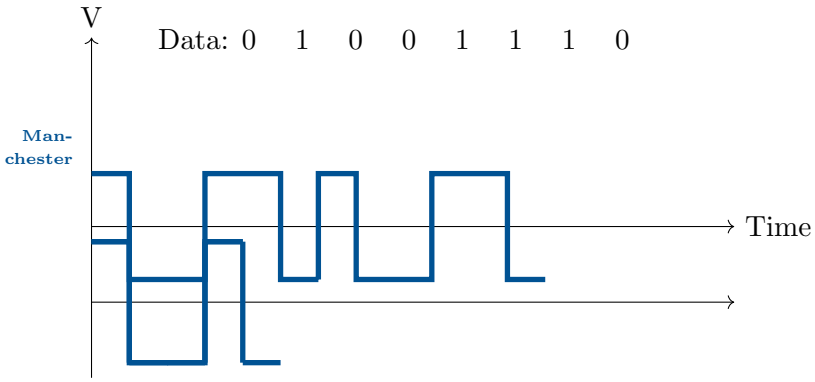
Differential Manchester Encoding

- **Always** has a transition at the **middle** of each bit period (for clock synchronization).
- **Binary 0** → Additional transition at the **beginning** of the bit period.
- **Binary 1** → **No** transition at the beginning of the bit period.

Advantages:

- More robust than Manchester: the receiver looks at the presence/absence of a beginning transition, not the direction. This makes it immune to polarity inversion on the cable!
- Also has excellent clock synchronization.

Used in: Token Ring networks.



Manchester: mid-bit transition always; 0 = high→low, 1 = low→high

1.5 Bipolar Schemes

Bipolar AMI (Alternate Mark Inversion)

In bipolar schemes, three voltage levels are used: **+V, 0, -V**.

- **Binary 0** → Zero voltage (0V).
- **Binary 1** → Alternates between +V and -V. First 1 = +V, next 1 = -V, next 1 = +V, etc.

Advantages:

- **No DC component**: Over time, equal numbers of +V and -V pulses, so average = 0.
- **Built-in error detection**: If you ever receive two consecutive 1s with the SAME polarity (both +V or both -V), you know an error occurred ("AMI violation").

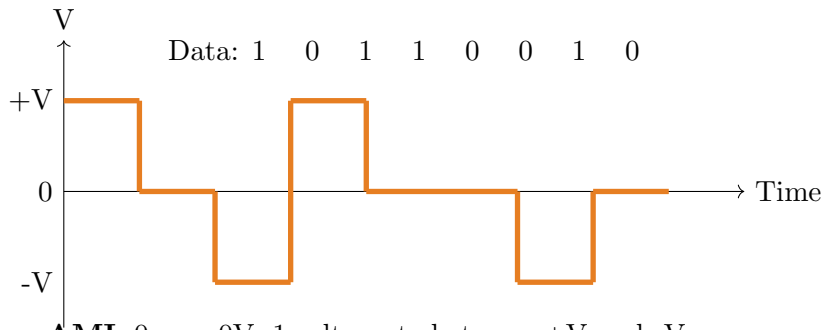
Disadvantage: Long strings of 0s = no transitions = synchronization problem.
Used in: T-carrier lines (T1, T3 telephone networks).

Pseudoternary

The inverse of AMI:

- **Binary 1** → Zero voltage (0V).
- **Binary 0** → Alternates between +V and -V.

Same properties as AMI but with roles of 0 and 1 swapped.



AMI: 0s are 0V, 1s alternate between +V and -V

1.6 Multilevel Schemes (Brief Theory)

Why Multilevel Schemes?

The idea behind multilevel schemes is to encode **multiple bits into one signal element**, achieving higher data rates without increasing the bandwidth.
The naming convention is **mBnL**:

- **m**: number of input binary bits
- **B**: binary (the input is binary)
- **n**: number of output signal elements
- **L**: signal levels (2=binary, T=ternary, Q=quaternary)

Examples: 2B1Q (2 bits mapped to 1 quaternary signal), 8B6T (8 bits to 6 ternary signals).

1.7 Key Comparison Table

Scheme	Levels	DC	Self-Sync	BW Need	Use
Unipolar NRZ	2	Yes	No	$N/2$	Labs only
NRZ-L	2	Possible	No	$N/2$	Simple links
NRZ-I	2	Possible	Partial	$N/2$	USB
RZ	3	No	Yes	N	
Manchester	2	No	Yes	N	Ethernet
Diff. Manchester	2	No	Yes	N	Token Ring
AMI	3	No	Partial	$N/2$	T1 lines

Chapter 2 Transmission Modes — Section 4.3

Transmission Mode: What Does It Mean?

When we send data from one device to another, we need to decide: **how are the bits arranged for transmission?** This is the question answered by the transmission mode. There are two fundamental choices:

1. **Parallel**: Send multiple bits simultaneously on multiple wires.
2. **Serial**: Send one bit at a time on a single wire.

Within serial transmission, there are two further choices:

- **Asynchronous serial**: No clock; timing embedded in start/stop bits.
- **Synchronous serial**: Both sides share a clock; data sent in continuous frames.

2.1 Parallel Transmission

Parallel Transmission

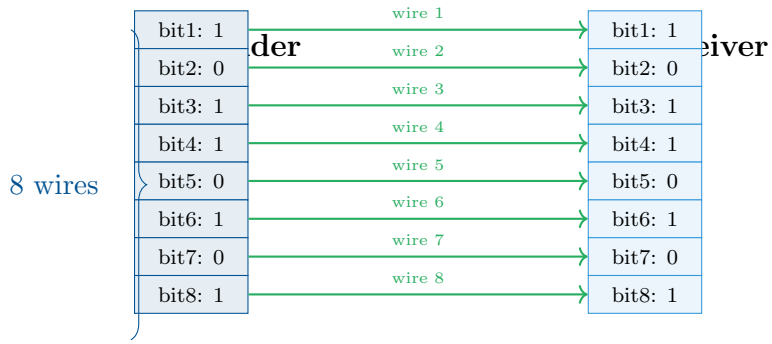
In parallel transmission, we send n **bits at a time** using n separate wires simultaneously. Each wire carries 1 bit, so a group of bits (like a byte = 8 bits) travels side-by-side.

Advantages:

- **Speed**: 8 wires send a byte in 1 clock cycle; serial needs 8 clock cycles.
- **Simple timing**: Just one clock pulse per byte.

Disadvantages:

- **Cost**: Requires n wires, n transmitters, n receivers. Expensive!
- **Distance limit**: Longer cables are prone to **skew** (bits on different wires arriving at slightly different times). At high speeds over long distances, skew becomes a serious problem.
- **Practical limit**: Used only for short distances (e.g., within a computer motherboard, connecting to an old printer via parallel port, inside CPUs).



Parallel Transmission: 8 bits sent simultaneously on 8 wires

2.2 Serial Transmission

Serial Transmission

In serial transmission, bits are sent **one at a time** over a single channel. This eliminates the cost and skew problems of parallel transmission. All bits travel the same path, so they arrive in the same order they were sent, one after another.
Advantages:

- **Cheap**: Only one wire/channel needed.
- **Long distance**: No skew problem. Works across continents.
- **Reliable**: All bits take the same path.

Disadvantage:

- **Slower per clock cycle** than parallel (one bit at a time).
- But modern serial links can run at very high speeds (USB 3.2 = 20 Gbps serial!).

Used in: USB, HDMI, PCIe, Ethernet, Wi-Fi, optical fiber — essentially all networking!

2.2.1 Asynchronous Serial Transmission

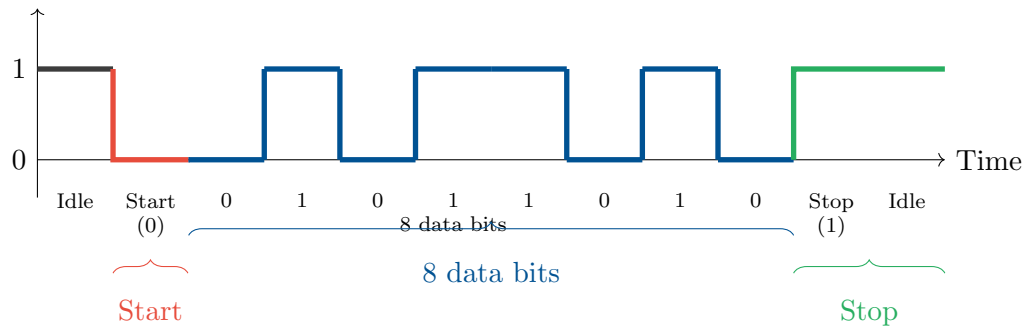
Asynchronous Transmission — "Start and Stop" Method

The word "asynchronous" means **"without synchronization."** In this mode:

- **No shared clock** between sender and receiver.
- Each group of bits (usually 1 byte = 8 bits) is wrapped with a **start bit** and one or two **stop bits**.
- The **start bit** (always a 0) signals "here comes a byte!" It alerts the receiver to activate its local clock and start reading.
- The **stop bit(s)** (always 1 or 11) signal "byte is complete." The receiver deactivates its clock.

- Between bytes, the line is **idle** (held at logical 1, i.e., the "mark" state).

How Asynchronous Transmission Works



Overhead of Asynchronous Transmission

Calculating Overhead in Async Transmission

Setup: 1 start bit + 8 data bits + 1 stop bit per byte.
Total bits per byte: 10 bits to send 8 data bits.
Overhead:

$$\text{Overhead} = \frac{10-8}{8} \times 100\% = \frac{2}{8} \times 100\% = \mathbf{25}\%$$

For every 8 data bits, 2 extra bits (25% overhead) are used for synchronization.

Advantages and Disadvantages

Advantages	Disadvantages
Simple & cheap (no shared clock)	25% overhead (wastes bandwidth)
Works even if characters are sent at irregular intervals (e.g., typing on a keyboard)	Slow (extra bits for each byte)
No need to continuously synchronize	Only suitable for slow, bursty communication

Used in: RS-232 serial ports, older modems, terminal communication, some sensor networks.

2.2.2 Synchronous Serial Transmission

Synchronous Transmission — "Frame-Based" Method

In synchronous transmission:

- **No start/stop bits per character.** Instead, bits are sent as a continuous stream.
- The continuous stream is organized into **frames** (large groups of bits).

- The sender and receiver share a **clock** (either explicitly via a clock wire, or the clock is embedded in the signal itself using schemes like Manchester encoding).
- The receiver reads bits in sync with the clock, decoding the data from the continuous stream.
- Special **synchronization patterns** (sync bytes like `01111110`) are placed at the start of frames so the receiver knows where a frame begins.

Synchronous: continuous bit stream organized into frames; no per-byte start/stop bits

Advantages and Disadvantages

Advantages	Disadvantages
Very low overhead (no per-byte start/stop bits)	Requires precise timing (shared clock or self-sync signal)
High efficiency for large data transfers	More complex hardware
Supports very high data rates	If sync is lost, entire frame is corrupted

Used in: Ethernet, Wi-Fi, USB (at high speeds), HDLC, SONET/SDH (fiber networks).

2.3 Comparison: Parallel vs. Serial vs. Async vs. Sync

Property	Parallel	Serial (Async)	Serial (Sync)
Wires needed	n (many)	1	1
Speed	High (per cycle)	Lower	Very High
Distance	Short (skew)	Long	Long
Overhead	None	25%+	≈0%
Cost	High	Low	Medium
Synchronization	Easy	Via start/stop bits	Clock or embedded
Use case	Within devices	Keyboards, modems	Networks, high-speed

Chapter Summary

Key Takeaways

Section 4.1.1 — Line Coding (Theory):

- Line coding converts digital data (bits) into digital signals for transmission.
- Key characteristics: DC component, baseline wandering, self-synchronization, signal rate.
- **Unipolar NRZ**: Simple but has DC component and no sync.
- **NRZ-L**: Level determines bit; **NRZ-I**: Transition (inversion) on 1.
- **RZ**: Returns to 0 mid-bit; great sync but needs double bandwidth.
- **Manchester**: Mid-bit transition always; 0=H→L, 1=L→H. Used in Ethernet.
- **Differential Manchester**: Mid-bit transition always; beginning transition = 0. Used in Token Ring.
- **AMI**: 0=0V, 1s alternate +V/-V. No DC, built-in error detection.

Section 4.3 — Transmission Modes:

- **Parallel**: Many bits simultaneously; fast but expensive, short-distance only (skew).
- **Serial Async**: Start/stop bits per character; simple, 25% overhead, slow.
- **Serial Sync**: Continuous stream with frames; low overhead, high speed, needs clock.
- Almost all networking uses **synchronous serial** transmission.

Data Communications & Networks

Chapter 6: Sections 6.1 & 6.2

Bandwidth Utilization: Multiplexing & Spread Spectrum

Topics Covered:

- 6.1.1 Frequency-Division Multiplexing (FDM)
- 6.1.2 Wavelength-Division Multiplexing (WDM)
- 6.1.3 Time-Division Multiplexing (TDM): Sync & Statistical
- 6.2.1 Frequency Hopping Spread Spectrum (FHSS)
- 6.2.2 Direct Sequence Spread Spectrum (DSSS)

Reference: Forouzan, "Data Communications and Networking," 5th ed.

April 11, 2026

Contents

1 Multiplexing — Section 6.1 **2**

1.1 Frequency-Division Multiplexing (FDM) 2

1.1.1 Core Concept 3

1.1.2 Numerical Examples 3

1.1.3 Analog Telephone Hierarchy 5

1.2 Wavelength-Division Multiplexing (WDM) 5

1.3 Time-Division Multiplexing (TDM) 6

1.3.1 Core Concept 6

1.3.2 Synchronous TDM 6

1.3.3 Digital Signal (DS) / T-Line Hierarchy 8

1.3.4 Statistical TDM 9

2 Spread Spectrum — Section 6.2 **10**

2.1 Frequency Hopping Spread Spectrum (FHSS) 10

2.1.1 How FHSS Works 10

2.1.2 FHSS Bandwidth 11

2.2 Direct Sequence Spread Spectrum (DSSS) 12

2.2.1 How DSSS Works 12

2.2.2 Comparison: FHSS vs DSSS 13

Chapter 1 Multiplexing — Section 6.1

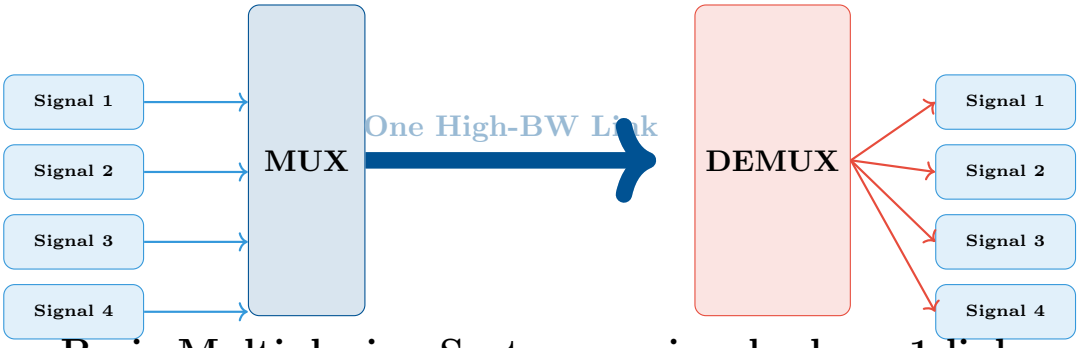
The Problem Multiplexing Solves

Imagine you own an expensive 8-lane highway (high-bandwidth fiber cable), but you only need to send one car (one signal). That's wasteful! **Multiplexing** is the engineering solution: it lets multiple signals *share* one high-bandwidth channel simultaneously.

Why do we need multiplexing?

- Installing a separate cable for each pair of users is astronomically expensive.
- Today's transmission media (fiber optics, satellite) have enormous bandwidth — far more than any single signal needs.
- Multiplexing packs many signals onto one link, maximizing efficiency and slashing cost.

In a multiplexed system, n signals share the bandwidth of one link. A **multiplexer (MUX)** combines them at the sender side; a **demultiplexer (DEMUX)** splits them at the receiver side.



Basic Multiplexing System: n signals share 1 link

1.1 Frequency-Division Multiplexing (FDM)

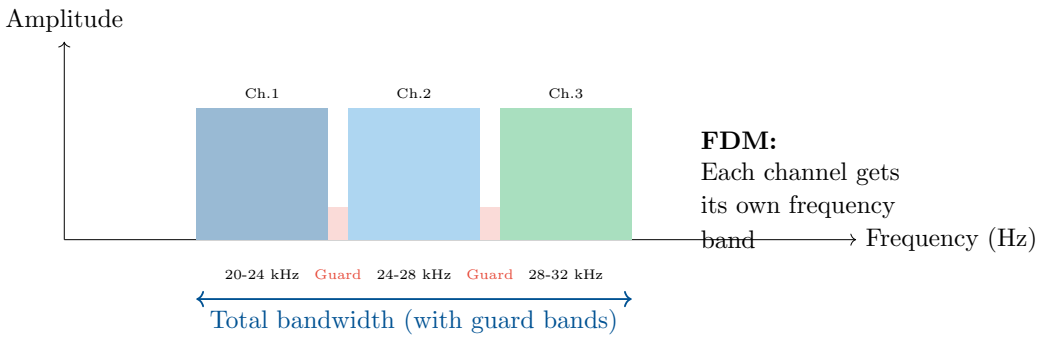
1.1.1 Core Concept

FDM: Splitting the Frequency Spectrum

FDM is an *analog* technique. Think of it like a radio dial: many stations broadcast simultaneously on the same air, each on a *different frequency*. You tune to one frequency to get one station.
How it works:

1. Each input signal occupies a **base bandwidth** (e.g., 4 kHz for voice).
2. Each signal is **modulated** onto a different carrier frequency: $f_1, f_2, f_3, \ldots$
3. The modulated signals are **combined** (added together) into one composite signal.
4. This composite signal is sent over the shared link.
5. At the receiver, **bandpass filters** extract each individual signal.
6. Each filtered signal is **demodulated** to recover the original.

Guard bands: Thin strips of unused spectrum are placed between channels to prevent them from overlapping (interference).



1.1.2 Numerical Examples

Example 6.1: Three 4-kHz Voice Channels via FDM

Given: 3 voice channels, each with 4 kHz bandwidth. Available link: 20–32 kHz. No guard bands. Show the FDM configuration.

Solution:

- Total available bandwidth: $32 - 20 = 12$ kHz.
- Each channel needs 4 kHz $\Rightarrow 3 \times 4 = 12$ kHz. Perfect fit!
- Channel 1: modulated to occupy **20–24 kHz** (carrier $f_1 = 22$ kHz center)
- Channel 2: modulated to occupy **24–28 kHz** (carrier $f_2 = 26$ kHz center)
- Channel 3: modulated to occupy **28–32 kHz** (carrier $f_3 = 30$ kHz center)

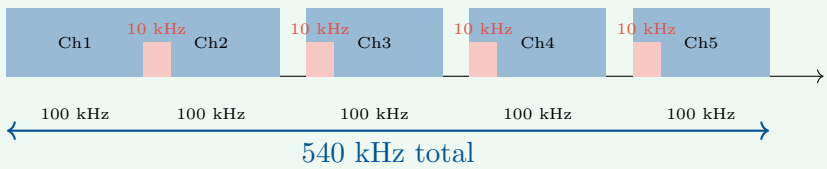
At the receiver: filter 1 passes 20–24 kHz, filter 2 passes 24–28 kHz, filter 3 passes 28–32 kHz. Each filtered signal is then shifted back to 0–4 kHz by demodulation.

Example 6.2: Guard Bands Required

Given: 5 channels, each with 100 kHz bandwidth. Guard band = 10 kHz between channels. Find the minimum total link bandwidth.

Solution:
With 5 channels, there are **4 gaps** between them (between channels 1–2, 2–3, 3–4, 4–5):

$$\text{Total BW} = 5 \times 100 \text{ kHz} + 4 \times 10 \text{ kHz} = 500 + 40 = \mathbf{540} \text{ kHz}$$



Example 6.3: Digital Sources + FDM via Modulation

Given: 4 digital channels, each at 1 Mbps, using a satellite analog channel of 1 MHz total. Design an FDM configuration.

Solution:

- Divide the 1 MHz satellite channel into 4 sub-channels of $\frac{1000}{4} = \mathbf{250}$ kHz each.
- Each 1 Mbps digital source is modulated using **16-QAM** (which encodes 4 bits per symbol).
- With 16-QAM, 1 Mbps digital data needs: $\frac{1 \text{ Mbps}}{4 \text{ bits/symbol}} = 250{,}000$ baud $=$ 250 kHz. Fits perfectly!

Example 6.4: AMPS Cellular FDM (Real World)

Given: AMPS uses band 824–849 MHz (sending) and 869–894 MHz (receiving). Each user gets a 30 kHz channel. How many simultaneous users?

Solution:

$$\text{Band width} = 849 - 824 = 25 \text{ MHz}$$

$$\text{Number of channels} = \frac{25 \text{ MHz}}{30 \text{ kHz}} = \frac{25{,}000 \text{ kHz}}{30 \text{ kHz}} = 833.3 \approx 832 \text{ channels}$$

Of 832 channels, 42 are for control $\Rightarrow$ **790 channels for voice users**.

1.1.3 Analog Telephone Hierarchy

The telephone system uses a structured FDM hierarchy to combine thousands of calls:

Level	Name	Bandwidth	Voice Channels	Composition
Base	Voice	4 kHz	1	—
1	Group	48 kHz	12	12 voice
2	Supergroup	240 kHz	60	5 groups
3	Master group	2.52 MHz	600	10 supergroups
4	Jumbo group	16.984 MHz	3600	6 master groups

Real-World FDM Applications:

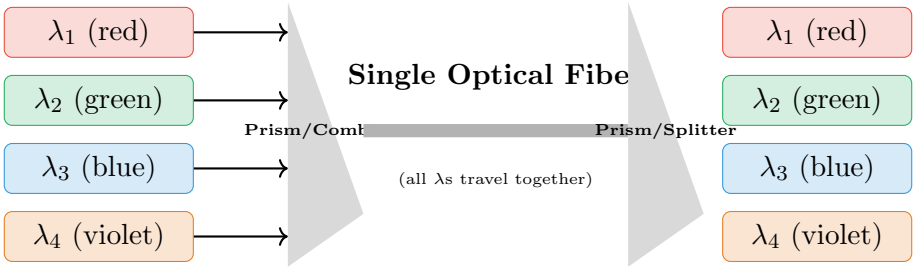
- **AM radio**: 530–1700 kHz band; each station uses 10 kHz.
- **FM radio**: 88–108 MHz band; each station uses 200 kHz.
- **TV channels**: Each channel has 6 MHz bandwidth.
- **1st-gen cellular (AMPS)**: 30 kHz per user channel (FDM in air).
- **DSL**: Splits phone line into voice (low frequencies) and data (high frequencies).

1.2 Wavelength-Division Multiplexing (WDM)

WDM: FDM for Light

WDM is essentially FDM for **optical fiber**. Instead of using different radio frequencies, WDM uses different **wavelengths of light** (colors). Each "color" carries a separate signal.

- Optical prisms (or diffraction gratings) combine and separate different light wavelengths.
- Each wavelength can carry a very high data rate (e.g., 10 Gbps each).
- **DWDM (Dense WDM)**: Can multiplex 32, 64, or even 160 wavelengths on one fiber.
- Used in long-haul internet backbone (trans-oceanic cables).



1.3 Time-Division Multiplexing (TDM)

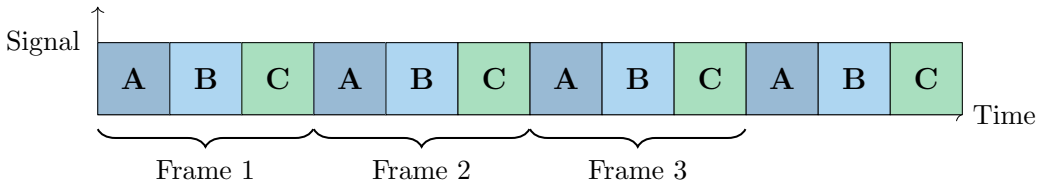
1.3.1 Core Concept

TDM: Sharing Time Instead of Frequency

While FDM divides the frequency spectrum, **TDM divides time**. Each input signal gets the *entire bandwidth of the link*, but only for a brief **time slot**. The multiplexer rapidly cycles through all input channels, giving each a turn on the link. The channel is shared **in time**.

Analogy: One teacher (link) teaches 30 students (channels). In FDM, each student gets their own quiet corner (separate frequency). In TDM, each student gets the teacher's full attention for 2 minutes, then the next student's turn comes. The teacher rotates rapidly enough that everyone is served effectively.

TDM is a **digital** technique — it works with discrete data (bits, characters, blocks).



Each frame = 1 slot for each channel. Channel A, B, C share the link in time.

1.3.2 Synchronous TDM

Synchronous TDM

In **synchronous TDM**:

- Each input gets a **fixed, reserved time slot** in every frame — even if it has no data to send!
- A **frame** = one complete cycle of all n time slots.
- If input unit duration = T s, the output time slot = T/n s (n times shorter).
- Output data rate = $n\times$ input data rate (each of n inputs contributes equally).

Key relationships:

$$\text{Output frame rate} = \text{Input data rate (frames per second)}$$
$$\text{Output bit rate} = n \times \text{Input bit rate}$$
$$\text{Frame duration} = \frac{1}{\text{Frame rate}} = \frac{1}{\text{Input data rate}}$$

Worked Numerical Examples

Example 6.6: Four 1-Mbps channels multiplexed

Given: 4 input channels, each at 1 Mbps. Unit = 1 bit. Find: (1) Input bit duration, (2) Output bit duration, (3) Output bit rate, (4) Frame rate.

Solution:

1. **Input bit duration:** $T_{in} = \frac{1}{1 \text{ Mbps}} = 1\ \mu\text{s}$ per bit.
2. **Output bit duration:** The output slot is $n = 4$ times shorter.
$$T_{out} = \frac{T_{in}}{n} = \frac{1\ \mu\text{s}}{4} = \mathbf{0.25}\ \mu\text{s} = \mathbf{250}\ \text{ns}$$
3. **Output bit rate:** $N_{out} = \frac{1}{T_{out}} = \frac{1}{0.25\ \mu\text{s}} = \mathbf{4}$ Mbps
Or simply: $N_{out} = n \times N_{in} = 4 \times 1$ Mbps $= 4$ Mbps.
4. **Frame rate:** Frame rate = input data rate (each input contributes 1 bit per frame).
$$\text{Frame rate} = 1{,}000{,}000 \text{ frames/s} = \mathbf{1} \text{ Mframes/s}$$
Verify: 1,000,000 frames/s × 4 bits/frame = 4 Mbps. ✓

Example 6.7: Four 1-kbps channels multiplexed

Given: 4 connections at 1 kbps each. Unit = 1 bit.

Solution:

$$\text{Output bit rate} = 4 \times 1 \text{ kbps} = \mathbf{4} \text{ kbps}$$
$$\text{Output bit duration} = \frac{1}{4 \text{ kbps}} = \mathbf{0.25} \text{ ms}$$
$$\text{Frame rate} = 1{,}000 \text{ frames/s (same as input rate)}$$
$$\text{Frame duration} = \frac{1}{1000} = \mathbf{1} \text{ ms}$$

Example 6.8: Byte interleaving — 100 bytes/s per channel

Given: 4 channels, each sending 100 bytes/s. Interleaved unit = 1 byte (8 bits).

Solution:

- Frame size: 4 channels $\times$ 1 byte = **4 bytes = 32 bits per frame**.
- Frame rate: Each channel sends 100 bytes/s, so there are **100 frames/s**.
- Frame duration: $\frac{1}{100}$ = **0.01** s = **10** ms.
- Link bit rate: 100 frames/s $\times$ 32 bits/frame = **3200** bps.
- Check: $4 \times 100 \times 8 = 3200$ bps. ✓

Example 6.9: Four 100-kbps channels with bit interleaving

Given: 4 channels, each at 100 kbps. Unit = 1 bit. Find frame details.

Solution:

- Output bit rate: 4×100 kbps = **400** kbps.
- Frame rate = input data rate = **100,000** frames/s.
- Frame size = 4 bits (1 from each channel).
- Frame duration: $\frac{1}{100{,}000}$ = **10** μs.
- Verify: $100{,}000 \times 4 = 400{,}000$ bps. ✓

Example 6.10: Framing bits (synchronization bits)

Given: 4 sources, each creating 250 characters/s. 1 synchronization bit added per frame. Interleaved unit = 1 character = 8 bits.

Solution:

1. **Data rate of each source:** 250 chars/s$\times$8 bits/char = **2000** bps = **2** kbps.
2. **Character duration (each source):** $\frac{1}{250}$ = **4** ms.
3. **Frame rate:** Frame rate = character rate = **250** frames/s.
4. **Frame duration:** $\frac{1}{250}$ = **4** ms.
5. **Bits per frame:** 4 sources $\times$ 8 bits + 1 sync bit = **33** bits.
6. **Link data rate:** 250 frames/s $\times$ 33 bits/frame = **8250** bps.

1.3.3 Digital Signal (DS) / T-Line Hierarchy

Telephone companies implement TDM via a standardized hierarchy:

Service	T-Line	Data Rate (Mbps)	Voice Channels
DS-0	—	0.064 (64 kbps)	1
DS-1	T-1	1.544	24
DS-2	T-2	6.312	96
DS-3	T-3	44.736	672
DS-4	T-4	274.176	4032

T-1 Frame Structure Verification

T-1 Frame: 24 channels × 8 bits + 1 framing bit = $\mathbf{24 \times 8 + 1 = 193}$ bits per frame.
Frame rate: Telephone systems sample voice at 8000 Hz (Nyquist for 4-kHz voice bandwidth), so there are **8000** frames/s.
T-1 Data Rate:

$$8000 \text{ frames/s} \times 193 \text{ bits/frame} = \mathbf{1{,}544{,}000} \text{ bps} = \mathbf{1.544} \text{ Mbps}$$

This is the famous T-1 rate!

1.3.4 Statistical TDM

Statistical TDM: Smarter Allocation

Problem with synchronous TDM: Reserved empty slots are wasteful. If a terminal is idle, its slot in every frame is empty.
Statistical TDM solution:

- Slots are **not pre-assigned**. They are allocated *dynamically* — only to sources that have data.
- Empty slots are eliminated; the output frame is **filled with actual data** from active inputs.
- Each slot carries an **address** identifying which channel the data belongs to (since positions are no longer fixed).
- If n sources are active simultaneously, the output rate must accommodate all.
- **Output rate < sum of input rates** in statistical TDM (statistical multiplexing gain), because not all sources are active all the time.

Used in: Internet routers (packet switching is essentially statistical TDM!).

Feature	Synchronous TDM	Statistical TDM
Slot assignment	Fixed per channel	Dynamic (on-demand)
Empty slots	Yes (idle channels waste space)	No (slots always carry data)
Address in slot	Not needed (position = channel)	Required (who does this data belong to?)
Output data rate	= sum of all input rates	< sum of all input rates
Efficiency	Lower	Higher
Complexity	Simple	More complex
Use	T-1 lines	ATM, internet packets

Chapter 2 Spread Spectrum — Section 6.2

Why Spread Spectrum?

Multiplexing (Section 6.1) was about *efficiency*: packing more signals into a link. Spread spectrum is about something different: **security and anti-jamming**.
The core idea: Deliberately *expand* (spread) the bandwidth of a signal far beyond what is strictly needed to carry the data.
Why would anyone spread bandwidth on purpose? Two reasons:

1. **Privacy/Security:** A jammer (enemy, eavesdropper) doesn't know which frequency the signal is using, so jamming or interception is very hard.
2. **Multiple access:** Multiple users can share the same wide bandwidth without interfering, if each uses a unique spreading code (used in CDMA cellular phones).

Requirements:

1. The spread bandwidth B_{SS} must be much greater than the original bandwidth B.
2. The spreading process must be *independent* of the original signal (done via a separate spreading code).

2.1 Frequency Hopping Spread Spectrum (FHSS)

2.1.1 How FHSS Works

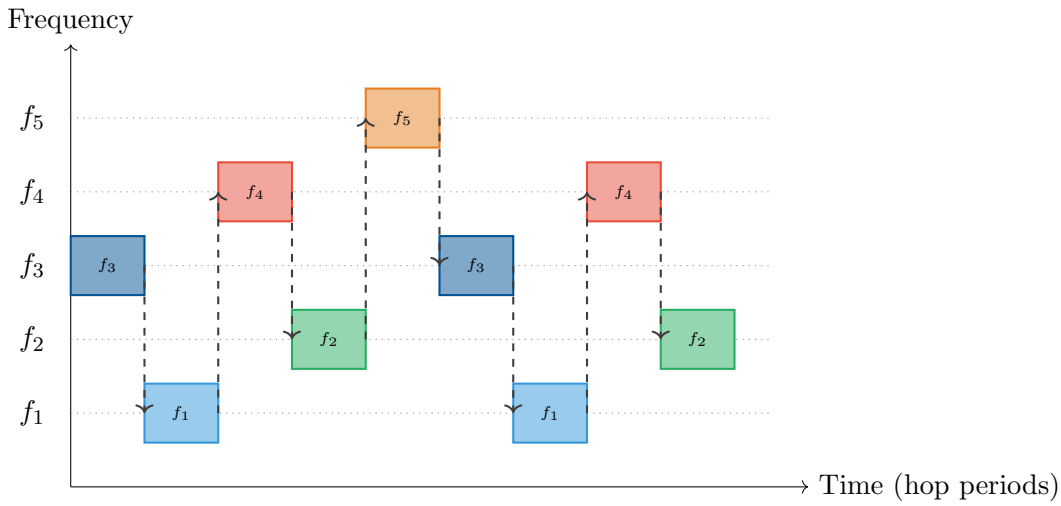
FHSS: Jumping Between Frequencies

In FHSS, the signal doesn't stay on one carrier frequency. Instead, it rapidly **"hops"** from one carrier frequency to another, following a **pseudorandom sequence** known only to the sender and receiver.
Process:

1. The sender has a **frequency table** and a **pseudorandom code generator**. The code determines which frequency to use in each **hop period**.
2. The original signal is modulated onto the selected frequency for one hop

period.

3. After the hop period ends, the sender moves to the next frequency in the sequence.
4. The receiver, using the same code and synchronized with the sender, knows exactly which frequency to tune to at each moment.
5. An eavesdropper doesn't know the hopping sequence, so they can't follow the signal.
6. A jammer can only jam one frequency at a time; since the signal hops away, only a few bits are lost.



Signal hops between frequencies each period
Eavesdropper cannot predict the next frequency

2.1.2 FHSS Bandwidth

FHSS Bandwidth

If the signal hops among k frequencies, and the original signal bandwidth is B:

$$B_{FHSS} \approx k \times B_{hop}$$

Where B_{hop} is the bandwidth at each hopped frequency (approximately equal to the original signal bandwidth B). The total spread bandwidth can be many times the original.

Real-World FHSS Applications:

- **Bluetooth**: Uses 79 frequencies (2402–2480 MHz), hopping 1600 times/second.
- **Early 802.11 Wi-Fi**: Used FHSS in the 2.4 GHz band.
- **Military radios**: FHSS provides anti-jamming capability.

2.2 Direct Sequence Spread Spectrum (DSSS)

2.2.1 How DSSS Works

DSSS: Replacing One Bit with a Chip Sequence

In DSSS, each data bit is replaced with a **sequence of shorter chips**. The chip sequence is called the **spreading code** (or chipping code).
Process:

1. Original data bit (1 or -1 in polar NRZ) is multiplied by a **spreading code** (a sequence of n chips, each +1 or -1).
2. This **multiplication** produces a spread signal. If the bit is 1, the chip sequence is sent unchanged. If the bit is -1, the chip sequence is inverted.
3. The bandwidth increases by a factor of n (the **spreading factor**).
4. At the receiver, the same spreading code is used to **de-spread** the signal and recover the original bits.
5. An eavesdropper without the code sees only noise — they cannot recover the data.

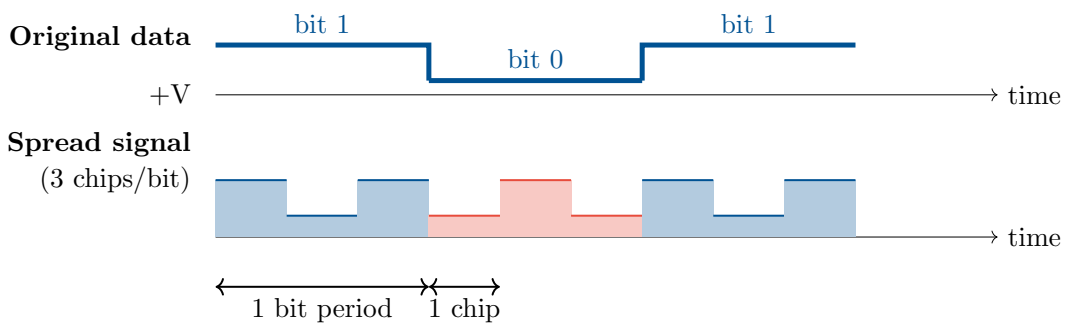
Example: DSSS with Barker Sequence (11 chips)

Given: Original data: bits 1, 0, 1. Barker sequence (11 chips, polar NRZ): $+1-1+1+1-1+1+1+1-1-1-1$ (an 11-chip sequence used in 802.11b Wi-Fi).

Encoding process:

- Bit 1 (= +1): transmitted chips = the Barker sequence itself: $+1-1+1+1-1+1+1+1-1-1-1$
- Bit 0 (= -1): transmitted chips = Barker sequence multiplied by -1: $-1+1-1-1+1-1-1-1+1+1+1$
- Bit 1 (= +1): again the Barker sequence: $+1-1+1+1-1+1+1+1-1-1-1$

Bandwidth change: Original 1 bit → 11 chips. Bandwidth is now 11 times larger!



Bandwidth × 3 (or ×11 for Barker sequence in real DSSS)

2.2.2 Comparison: FHSS vs DSSS

Feature	FHSS	DSSS
Spreading method	Changes carrier frequency	Replaces bits with chip sequence
Bandwidth expansion	Uses multiple channels	Multiplied by spreading factor
Anti-jamming	Excellent (moves away from jammer)	Good (signal looks like noise)
Privacy	Good	Good
Multiple access	Users use different hop sequences	Users use different codes (CDMA)
Used in	Bluetooth, early Wi-Fi	802.11b Wi-Fi, GPS, CDMA cellular

Chapter Summary

Key Takeaways — Chapter 6

Section 6.1 — Multiplexing:

- **FDM**: Analog signals share bandwidth by using different frequency bands. Guard bands prevent interference.
- Example: AM radio (10 kHz each), FM (200 kHz each), TV (6 MHz each), AMPS (30 kHz each).
- **WDM**: Like FDM but for light in optical fiber. Used in backbone internet.
- **Synchronous TDM**: Digital signals share time. Fixed slots per channel. Output rate = $n\times$ input rate. T-1 = 1.544 Mbps (24 channels $\times$ 64 kbps + framing).
- **Statistical TDM**: Dynamic slot allocation; no empty slots; needs address field; more efficient.

Section 6.2 — Spread Spectrum:

- **FHSS**: Signal hops among many frequencies using a pseudorandom sequence. Anti-jamming, privacy.
- **DSSS**: Each bit replaced by a chip sequence; bandwidth expands by spreading factor. 802.11b uses 11-chip Barker sequence.

Critical Formulas:

- FDM total bandwidth = $\sum$ channel bandwidths + $(n-1)\times$ guard band
- TDM output rate = $n\times$ input rate (synchronous)
- T-1 frame = $24 \times 8 + 1 = 193$ bits; 8000 frames/s = 1.544 Mbps

Data Communications & Networks

Chapter 8: Sections 8.2 & 8.3

Circuit-Switched Networks & Packet Switching

Topics Covered:

- 8.1 Introduction to Switching
- 8.2 Circuit-Switched Networks (3 Phases, Efficiency, Delay)
- 8.3 Packet Switching: Datagram & Virtual-Circuit Networks

Reference: Forouzan, "Data Communications and Networking," 5th ed.

April 11, 2026

Contents

1 Introduction to Switching **2**
1.1 Three Methods of Switching 2

2 Circuit-Switched Networks — Section 8.2 **3**
2.1 Three Phases of Circuit Switching 3
2.1.1 Phase 1: Connection Setup (Signaling) 4
2.1.2 Phase 2: Data Transfer 4
2.1.3 Phase 3: Teardown 4
2.2 Efficiency of Circuit Switching 5
2.3 Delay in Circuit-Switched Networks 6

3 Packet Switching — Section 8.3 **7**
3.1 Datagram Networks 7
3.1.1 How Datagram Networks Work 7
3.1.2 Routing Table in Datagram Networks 8
3.1.3 Delay in Datagram Networks 8
3.2 Virtual-Circuit Networks 9
3.2.1 The Hybrid Approach 9
3.2.2 Three Phases of Virtual-Circuit Networks 10
3.2.3 Delay in Virtual-Circuit Networks 10
3.3 Comparison: All Three Switching Methods 11

Chapter 1 Introduction to Switching

Why Switching?

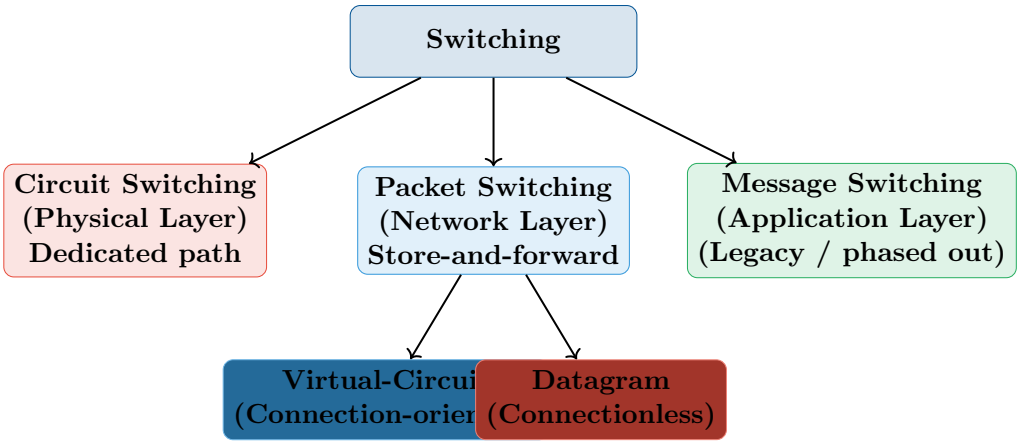
Imagine connecting every pair of computers in the world directly to each other with a dedicated wire. With $n = 7$ billion people, you'd need $\frac{n(n-1)}{2} \approx 2.45 \times 10^{19}$ wires — physically impossible!

Switching is the solution. Instead of a direct wire between every pair, we have a network of intermediate **switches**. A message is routed from source to destination through these switches.

A switched network:

- Consists of **switches** (routers, telephone exchange offices) connected by **links**.
- **End systems** (computers, phones) connect to switches.
- Data travels from source to destination via a path through multiple switches.

1.1 Three Methods of Switching



Chapter 2 Circuit-Switched Networks — Section 8.2

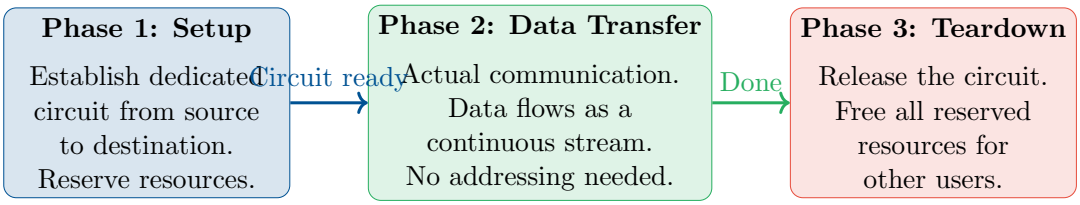
Circuit Switching: The Dedicated Highway

In circuit switching, before any data is transferred, a **dedicated path** (circuit) is established *end-to-end* between sender and receiver. This path remains reserved for the entire duration of the communication — even during silent periods.
Think of it like this: Booking a private taxi from city A to city Z. The taxi waits for you at every stop, nobody else can use it, and you have a guaranteed smooth ride.
Key characteristics:

- **Dedicated resources**: Bandwidth, switch buffers, processing time — all reserved for you.
- **Continuous flow**: Data flows as a stream, not packets. No packetization!
- **No addressing during transfer**: Once the circuit is set up, switches just forward the signal based on its time slot (TDM) or frequency band (FDM) — no address lookup needed!
- **Happens at physical layer**: Signals are simply routed, not processed.

2.1 Three Phases of Circuit Switching

Every circuit-switched communication requires **three distinct phases**:



2.1.1 Phase 1: Connection Setup (Signaling)

Setup Phase in Detail

1. Source end system sends a **setup request** message containing the destination address.
2. Each switch along the path receives the request, finds a free channel on its next link, **reserves it**, and forwards the request to the next switch.
3. Once the destination is reached, it sends an **acknowledgment (ACK)** back to the source.
4. Only after the ACK arrives at the source is the circuit considered established.
5. This involves **end-to-end addressing** (phone number, computer address).
6. During setup, each switch commits a dedicated channel (time slot in TDM, or frequency band in FDM) for this connection.

Important: During the setup phase, if any switch cannot allocate a channel, the connection is rejected (busy signal in telephone networks).

2.1.2 Phase 2: Data Transfer

Once the circuit is established:

- Data flows as a **continuous stream** (not packets).
- No routing decisions are made at intermediate switches. The path is fixed.
- No addressing in data frames. Switches forward based on physical channel allocation.
- Quality is **guaranteed**: fixed bandwidth, predictable delay, no competition.

2.1.3 Phase 3: Teardown

When communication ends:

- A disconnect signal is sent through the network.
- Each switch on the path **frees the reserved channel**.
- Resources become available for other connections.

2.2 Efficiency of Circuit Switching

Why Circuit Switching is Inefficient

The fundamental problem: **resources are reserved whether or not data is being sent**.
Consider a phone call:

- You speak for 30% of the call; the rest is silence (listening, pausing).
- During the 70% silence, the circuit is still fully reserved — nobody else can use it.
- This wastes valuable bandwidth.

In contrast, computer communication is extremely **bursty**: a web page request takes microseconds, then nothing for several seconds. A dedicated circuit during those idle seconds is completely wasteful.

Example 8.1: Simple Circuit-Switched Network for Telephone

Given: 8 telephones in a small area. Each telephone call uses a 4-kHz voice channel. Each link supports a maximum of 2 voice channels (using FDM). So each link has bandwidth $= 2 \times 4 = 8$ kHz.
Connections made:

- Phone 1 $\leftrightarrow$ Phone 7
- Phone 2 $\leftrightarrow$ Phone 5
- Phone 3 $\leftrightarrow$ Phone 8
- Phone 4 $\leftrightarrow$ Phone 6

Each connection uses one 4-kHz channel on each link it traverses. The switch controls which channel goes to which phone. When calls end, channels are released.
Key observation: If phones 1 and 7 hang up but phones 2 and 5 continue calling, the channels used by phones 1 and 7 become available for new calls — this is teardown in action.

Example 8.2: Company's Private Circuit-Switched Network

Given: A company has two offices connected by a T-1 line (1.544 Mbps). Each office has a 4×8 switch (4 inputs, 8 outputs). 4 output ports are looped back (internal communication); 4 output ports are for inter-office communication.
Design:

- Internal calls: Computers in the same office communicate through the 4 loopback ports.
- Inter-office calls: Use the T-1 line. With TDM, the T-1 can carry up to 24 voice channels or equivalent data channels between the two offices.

This is exactly how enterprise PBX (Private Branch Exchange) systems work!

2.3 Delay in Circuit-Switched Networks

Delay Analysis: Three Parts

Total delay in a circuit-switched network = Setup delay + Data transfer delay + Teardown delay.

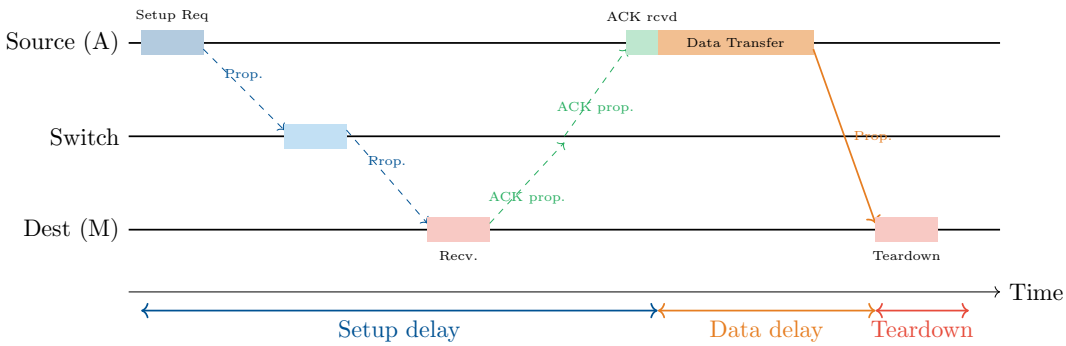
Setup delay has 4 parts:

1. *Propagation time* of setup request (source → destination).
2. *Transfer time* for the setup request itself.
3. *Propagation time* of acknowledgment (destination → source).
4. *Transfer time* for the acknowledgment.

Data transfer delay has 2 parts:

1. *Transmission time*: Time to put all the data bits onto the wire.
2. *Propagation time*: Time for the signal to travel from source to destination.

No queuing delay at intermediate switches: the path is dedicated, so data never waits! This is the key advantage of circuit switching for real-time applications like voice calls.



Chapter 3 Packet Switching — Section 8.3

Packet Switching: The Shared Road

Instead of a dedicated circuit, **packet switching** divides data into small chunks called **packets**. Each packet is sent independently and can take *any path* through the network.
The key revolution:

- No resources are pre-allocated. Packets share the network.
- Each packet carries **addressing information** (destination address in header).
- Switches (routers) make **routing decisions independently** for each packet.
- This is how the **Internet** works today!

Why "Store-and-Forward"? Each intermediate router must:

1. Receive (store) the entire packet.
2. Check it for errors.
3. Look up the routing table to find the next hop.
4. Forward the packet toward the destination.

3.1 Datagram Networks

3.1.1 How Datagram Networks Work

Datagram Approach: Connectionless

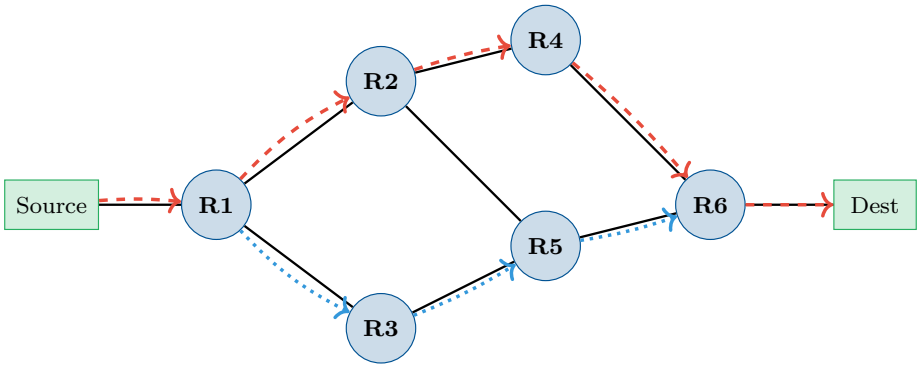
In a **datagram network**:

- **No connection setup phase**. Packets are sent immediately.
- Each packet is treated **independently**. Every router makes its own routing decision.
- Packets from the same source to the same destination may take **different paths**.
- Packets may arrive **out of order**. The receiving end must reorder them.

- Routing tables at each router are **dynamic** and can change.

The Internet is a datagram network.

Datagram Network: Each packet routed independently



Red dashed = Packet 1 path, Blue dotted = Packet 2 path (different routes!)

3.1.2 Routing Table in Datagram Networks

Each router maintains a **routing table** that maps destination addresses to outgoing interfaces:

Destination Address	Output Interface (Next Hop)
A	Interface 1
B	Interface 2
C	Interface 1
D	Interface 3
⋮	⋮

Datagram routing tables change dynamically. When a link fails or becomes congested, routers update their tables. This is why the Internet can route around failures — packets just take a different path!

3.1.3 Delay in Datagram Networks

Datagram Network Delay

Total Delay = (number of links + 1) $\times t_{trans}$ + propagation delays + queuing delays

For k routers (switches) and packet size L bits on links of bandwidth B bps:

$$\text{Processing delay per router} = 1 \times t_{trans} = \frac{L}{B}$$

Note: Each router must receive the ENTIRE packet before forwarding it (store-and-forward). This adds one transmission delay per router!

Example P8.2: Datagram Delay Calculation

Given: A message of 1000 bits is divided into 5 datagrams of 200 bits each. Each datagram crosses 3 links (2 routers), each link at 100 kbps. Assume zero propagation delay.

Transmission time per datagram per link:

$$t_{trans} = \frac{200 \text{ bits}}{100{,}000 \text{ bps}} = 2 \text{ ms}$$

Path: Source → Router1 → Router2 → Destination = 3 links.
For a single datagram:

$$\text{Delay} = 3 \times 2 \text{ ms} = 6 \text{ ms (store-and-forward at each link)}$$

For all 5 datagrams (pipelining effect):
With pipelining, after the first datagram reaches the destination at $t = 6$ ms, each subsequent datagram arrives 2 ms later:

$$\text{Last datagram arrives at} = 6 + 4 \times 2 = \mathbf{14} \text{ ms}$$

Without pipelining (wait for all to finish first): $5 \times 3 \times 2 = 30$ ms. Pipelining is much faster!

3.2 Virtual-Circuit Networks

3.2.1 The Hybrid Approach

Virtual-Circuit Networks: Best of Both Worlds

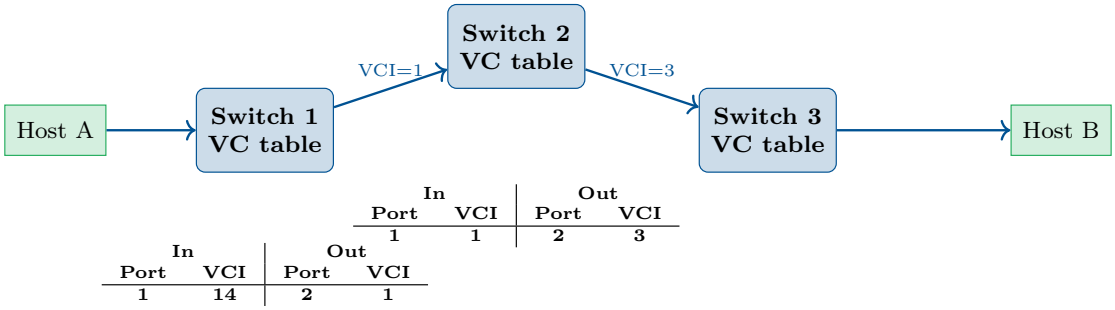
A virtual-circuit network combines features of both circuit switching and datagram networks:

- **Like circuit switching**: A path is established *before* data is sent (setup phase). All packets follow this **same path** in order.
- **Like datagram (packet switching)**: Data is still divided into **packets** (not a continuous stream). Resources are **NOT dedicated** — they are shared with other virtual circuits.

Virtual circuit identifier (VCI):

- Instead of a destination address in each packet, packets carry a small **VCI number**.
- The VCI changes at each switch (it's locally significant, not global).

- Routing tables map VCI-in to VCI-out and output interface.
- Since no destination address lookup is needed per packet, switching is **faster** than datagrams.



Virtual-Circuit: same path, VCI changes at each switch, packets go in order

3.2.2 Three Phases of Virtual-Circuit Networks

Like circuit switching, virtual-circuit networks have three phases:

Phase	Circuit Switching	Virtual Circuit	Datagram
Setup	Sets up circuit	Sets up VC path + VCI table	None
Data transfer	Continuous stream	Packets with VCI	Packets with dest. addr
Teardown	Releases resources	Removes VC entries	None

3.2.3 Delay in Virtual-Circuit Networks

Virtual-Circuit Delay

Virtual-circuit delay includes:

$$\text{Total Delay} = \underbrace{t_{setup}}_{\text{VC setup}} + \underbrace{k \times \frac{L}{B} + t_{prop}}_{\text{data transfer}} + \underbrace{t_{teardown}}_{\text{VC teardown}}$$

Key difference from datagrams:

- Virtual circuits have an **extra setup delay** that datagrams don't.
- But once the VC is set, routing is **faster** per packet (just look up VCI, not full address).
- For long communications with many packets, the VC setup overhead is amortized.

Example 8.1 (Circuit vs VC vs Datagram Delay)

Given: A message is 1000 bits. Sent over a network with 2 switches (3 links). Each link is 1 Mbps. Propagation delay per link = 1 ms. VC setup = 0.5 ms per switch.

Circuit switching delay:

$$t_{setup} \approx 2 \times (1 \text{ ms prop} + \text{small setup}) + \text{ACK} = \text{several ms}$$
$$t_{data} = \frac{1000}{1{,}000{,}000} \times 1 = 1 \text{ ms (transmission)} + 3 \text{ ms (3 links prop)} = 4 \text{ ms}$$

Datagram delay: (No setup; store-and-forward)

$$t_{data} = 3 \times \frac{1000}{1{,}000{,}000} + 3 \times 1 = 3 \text{ ms} + 3 \text{ ms} = \mathbf{6} \text{ ms}$$

Virtual circuit delay:

$$t_{setup} = 2 \times 0.5 = 1 \text{ ms (VC setup at 2 switches)}$$
$$t_{data} = 6 \text{ ms (same as datagram)}$$
$$\text{Total} = \mathbf{7} \text{ ms}$$

Conclusion: For a single message, datagrams are fastest. For many messages over same path, VCs amortize the setup cost.

3.3 Comparison: All Three Switching Methods

Feature	Circuit Switching	Virtual Circuit	Datagram
Path setup	Yes (before data)	Yes (before data)	No
Dedicated resources	Yes	No (shared)	No
Data type	Continuous stream	Packets (with VCI)	Packets (with addr.)
Addressing in data	No (circuit is fixed)	VCI (local)	Destination address
Packet order	Guaranteed	Guaranteed	Not guaranteed
Routing per packet	Not needed	VCI lookup (fast)	Full address lookup
Congestion during call	None (fixed BW)	Possible	Possible
Failure recovery	Must re-setup circuit	Must re-setup VC	Packets rerouted automatical
Bandwidth waste	Yes (idle time wasted)	No	No
Layer	Physical	Data-link / Network	Network
Example	PSTN phone network	ATM, Frame Relay	Internet (IP)

Chapter Summary

Key Takeaways — Chapter 8

Section 8.2 — Circuit Switching:

- Three phases: **Setup** → **Data Transfer** → **Teardown**.
- Resources (bandwidth, buffers) are **dedicated** for the entire call duration.
- Data flows as a **continuous stream**; no packetization; no addressing during transfer.
- **Efficient for voice** (real-time, continuous): guarantees bandwidth and low delay.
- **Inefficient for data** (bursty): wastes bandwidth during idle periods.
- No queuing delay at switches, but has setup delay.

Section 8.3 — Packet Switching:

- **Datagram**: No setup. Each packet routed independently. May arrive out of order. Internet uses this. Fast for short messages.
- **Virtual Circuit**: Setup phase establishes path. All packets follow same path (in order). Each packet carries VCI (not full address). Resources shared (not dedicated). ATM uses this.
- Both use **store-and-forward**: each router receives full packet before forwarding.
- **Datagram delay** $= k \times \frac{L}{B} +$ propagation (for k routers).
- Packet switching is far more **efficient** for bursty data (like internet traffic).

Data Communications & Networks

Chapter 9: Sections 9.1, 9.2.1 & 9.2.2

Introduction to Data-Link Layer & Link-Layer Addressing (ARP)

Topics Covered:

- 9.1 Introduction: Nodes, Links, Services, Sublayers
- 9.2.1 Link-Layer Addresses (MAC Addresses)
- 9.2.2 ARP: Address Resolution Protocol
  (ARP Request, Reply, Packet Format, & Packet Journey)

Reference: Forouzan, "Data Communications and Networking," 5th ed.

April 11, 2026

Contents

1 Introduction to Data-Link Layer — Section 9.1 **2**
1.1 Nodes and Links — Section 9.1.1 2
1.2 Services of the Data-Link Layer — Section 9.1.2 3
1.2.1 1. Framing 3
1.2.2 2. Flow Control 4
1.2.3 3. Error Control 4
1.2.4 4. Congestion Control 4
1.3 Two Categories of Links — Section 9.1.3 4
1.4 Two Sublayers — Section 9.1.4 5

2 Link-Layer Addressing — Section 9.2 **6**
2.1 Types of Link-Layer Addresses — Section 9.2.1 6
2.1.1 Unicast, Multicast, and Broadcast MAC Addresses 7
2.1.2 Why IP Addresses Remain the Same but MAC Addresses Change 8
2.2 Address Resolution Protocol (ARP) — Section 9.2.2 8
2.2.1 ARP Request and Reply 8
2.2.2 ARP Packet Format 9
2.2.3 Complete Packet Journey: Alice to Bob (Full Example) 10
2.2.4 ARP Caching 12

Chapter 1 Introduction to Data-Link Layer — Section 9.1

What Is the Data-Link Layer?

You already know the network layer (IP addresses, routing) gets a packet from source host to destination host. But how does the packet actually travel across each individual link between routers? That's the job of the **data-link layer**.
The data-link layer's domain:

- While the network layer handles the *end-to-end* journey (source host to destination host),
- the data-link layer handles *one hop at a time* — from one node to the very next node.
- "One hop" = one physical link (LAN cable, Wi-Fi, fiber, etc.).

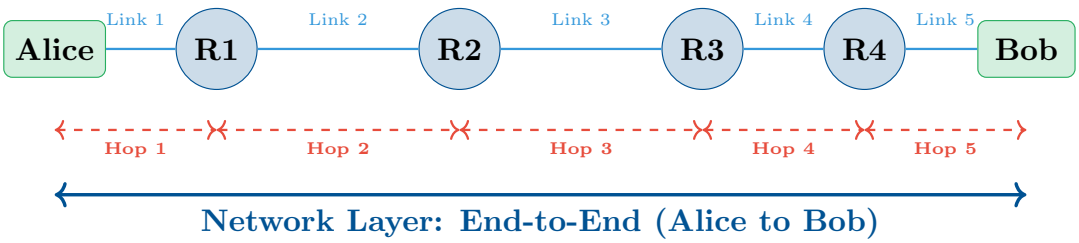
An analogy: Think of traveling from Lahore to London. The entire journey is the network layer's job. But the data-link layer only cares about one leg at a time: Lahore to Dubai airport (flight), then Dubai airport to London Heathrow (another flight). Each leg is one "hop."

1.1 Nodes and Links — Section 9.1.1

Nodes and Links Defined

In the context of the data-link layer:

- **Nodes** = hosts (computers, servers) and routers. These are the endpoints and intermediate processing points.
- **Links** = the physical networks connecting nodes. A LAN is a link, a WAN connection is a link.
- The data-link layer's job: deliver a frame from one node to the **adjacent** node on the same link.



Each data-link layer hop delivers a frame to the very next node

1.2 Services of the Data-Link Layer — Section 9.1.2

Four Services Provided by the Data-Link Layer

The data-link layer provides the following services to the network layer above it:

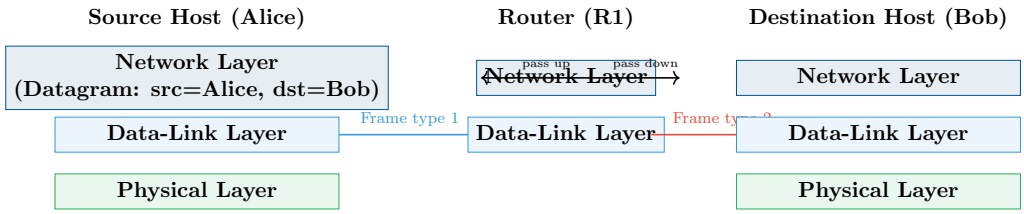
1.2.1 1. Framing

Framing: Packaging the Datagram

The data-link layer wraps the network-layer **datagram (packet)** in a **frame**.

- A frame = datagram + header (and sometimes a trailer).
- The header contains link-layer (MAC) addresses: destination and source.
- A frame is the unit of transmission at the data-link layer.
- Different links use different frame formats.

At the receiving node, the data-link layer **decapsulates** the datagram from the frame and passes it up to the network layer.
Why re-encapsulate at every router? Because different links may use completely different protocols and frame formats (Ethernet on Link 1, Wi-Fi on Link 2, fiber on Link 3). Each requires its own frame format!



Re-encapsulation at each router: same datagram, different frame each time

1.2.2 2. Flow Control

Flow Control: Matching Speeds

If the sender produces frames faster than the receiver can consume them:

- Buffer overflow at the receiver.
- Two solutions: **drop excess frames** or **feedback** to slow down the sender.
- The data-link layer can implement flow control (though TCP at transport layer also does this for end-to-end).

1.2.3 3. Error Control

Error Control: Detecting and Fixing Errors

Electromagnetic signals are susceptible to interference, attenuation, and noise. These cause bit errors in frames. The data-link layer must:

1. **Detect** errors (using checksums, CRC — studied in Chapter 10).
2. **Correct** errors at the receiver (forward error correction), OR
3. **Discard** corrupted frames and ask for retransmission.

Error control at the data-link layer is *per-link*. The transport layer (TCP) handles end-to-end reliability.

1.2.4 4. Congestion Control

Most data-link layer protocols do **not** directly handle congestion (that's the network and transport layers' job). However, some wide-area networks (e.g., Frame Relay) do implement basic congestion signaling.

1.3 Two Categories of Links — Section 9.1.3

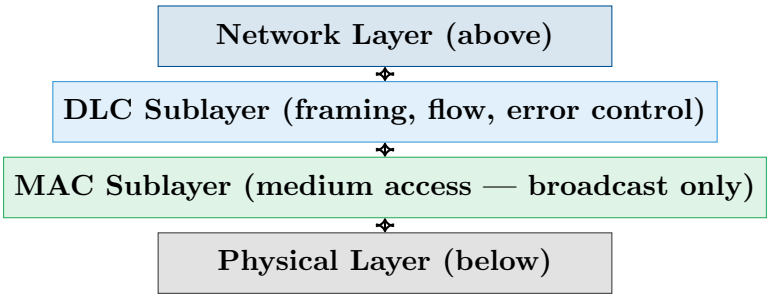
Feature	Point-to-Point Link	Broadcast Link
Shared?	No (dedicated to 2 devices)	Yes (shared among many)
Example	Fiber between two routers	Ethernet LAN, Wi-Fi
MAC needed?	Not always	Yes (multiple access control)
Who receives?	Only the connected device	All devices on the link

1.4 Two Sublayers — Section 9.1.4

DLC and MAC Sublayers

The data-link layer is divided into two sublayers:

1. **Data Link Control (DLC)**: Handles issues common to both point-to-point and broadcast links. Covers framing, flow control, and error control.
2. **Media Access Control (MAC)**: Handles issues specific to *broadcast* links — how multiple devices share a shared medium without collisions. This is what Ethernet (CSMA/CD), Wi-Fi (CSMA/CA), etc. address.



Chapter 2 Link-Layer Addressing — Section 9.2

Why Do We Need Link-Layer Addresses?

You might ask: we already have IP addresses (network layer). Why do we need *another* set of addresses at the data-link layer?

The problem:

- IP addresses define the *end-to-end* source and destination. These do NOT change as the packet travels through routers.
- But as a packet hops through routers, each hop uses a *different physical link*.
- Each link delivers the frame to the next router's specific interface — not the final destination.
- The IP address of Router R1 doesn't tell you *which wire* on that router's Ethernet card to send to!
- We need **link-layer addresses** (also called **MAC addresses** or **physical addresses**) to identify the specific hardware interface for each hop.

Rule:

- IP (network-layer) addresses: **stay the same** from source to destination. They are the "passport" of the packet.
- MAC (link-layer) addresses: **change at every hop**. They identify the next immediate destination on each link, like boarding passes that change at each airport.

2.1 Types of Link-Layer Addresses — Section 9.2.1

2.1.1 Unicast, Multicast, and Broadcast MAC Addresses

Three Types of Link-Layer Addresses

1. **Unicast address**: Identifies a *single specific* network interface (one device). Every NIC (Network Interface Card) has a globally unique unicast MAC address. Example: `3A:1B:4C:2D:5E:6F`. The LSB (least significant bit) of the first byte is 0.

2. **Multicast address**: A special address that a *group* of interfaces can belong to. A packet with a multicast destination is delivered to all members of that group on the link. The LSB of the first byte is 1 (to distinguish from unicast). Example: `01:00:5E:xx:xx:xx` (for IPv4 multicast on Ethernet).

3. **Broadcast address**: The "send to everyone" address. A frame with this destination is delivered to *all* devices on the link. Ethernet broadcast: `FF:FF:FF:FF:FF:FF` (all 48 bits = 1).

MAC Address Structure

bit 0 = I/G (Unicast/Multicast) bit 1 = U/L (Universal/Local)

OUI (24 bits): Manufacturer ID	Serial Number (24 bits): Device ID

← 48 bits total = 6 bytes = 12 hex digits (e.g., AA:BB:CC:DD:EE:FF)

Example 9.1: Unicast MAC Address

Question: In Ethernet (most common LAN), link-layer addresses are 48 bits long, expressed as 12 hexadecimal digits. Is the address `3A:1B:4C:2D:5E:6F` a unicast, multicast, or broadcast address?

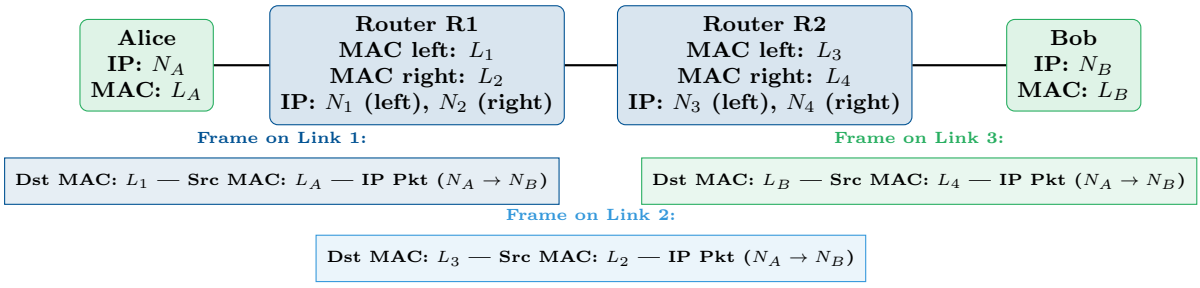
Solution:

- First byte in hex: `3A` = 0011 1010 in binary.
- Least significant bit (LSB) = 0 ⇒ this is a **unicast** address.

Example 9.2: Ethernet Multicast

Multicast addresses in Ethernet have an *odd first byte*. For example: `01:00:5E:XX:XX:XX` (first byte = `01` = 0000 0001; LSB = 1 ⇒ multicast).
Devices that want to receive multicast traffic must register their network interface to "listen" on the multicast MAC address.

2.1.2 Why IP Addresses Remain the Same but MAC Addresses Change



IP addresses ($N_A \to N_B$): UNCHANGED throughout the journey.
MAC addresses: CHANGE at every hop (destination MAC = next-hop MAC).

2.2 Address Resolution Protocol (ARP) — Section 9.2.2

The Problem ARP Solves

When a node (host or router) needs to send a frame to the **next hop**, it knows:

- The **IP address** of the next hop (from the routing/forwarding table).
- But **NOT the MAC address** of the next hop!

To create the frame, it needs the destination MAC address.
ARP (Address Resolution Protocol) is the mechanism that finds the MAC address corresponding to a known IP address.
ARP Slogan: "I know your IP address. What is your MAC address?"

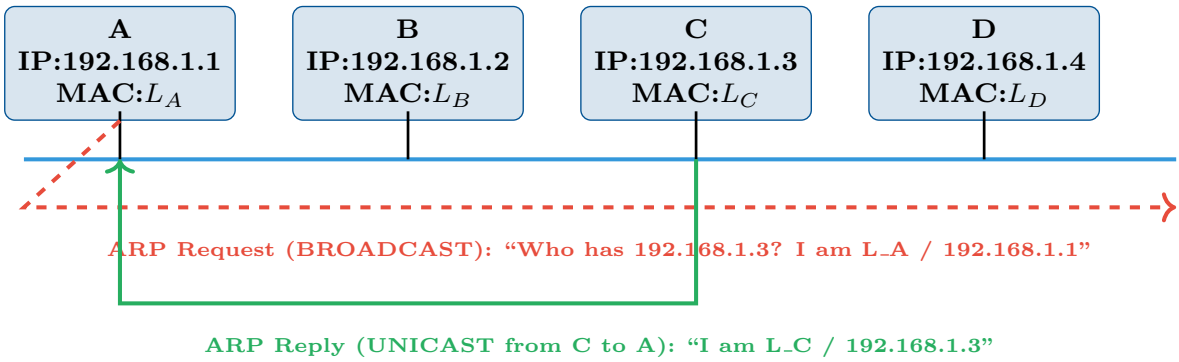
2.2.1 ARP Request and Reply

How ARP Works: Step by Step

1. **ARP Request (Broadcast):**
   - The asking node broadcasts an ARP request frame to all devices on the link.
   - Destination MAC in the frame = `FF:FF:FF:FF:FF:FF` (broadcast).
   - The ARP packet contains: "I am MAC L_A, IP N_A. Who has IP N_X? Please tell me your MAC."
   - **Every device** on the link receives this broadcast and checks if the target IP matches their own.
2. **ARP Reply (Unicast):**
   - Only the device with IP N_X responds.

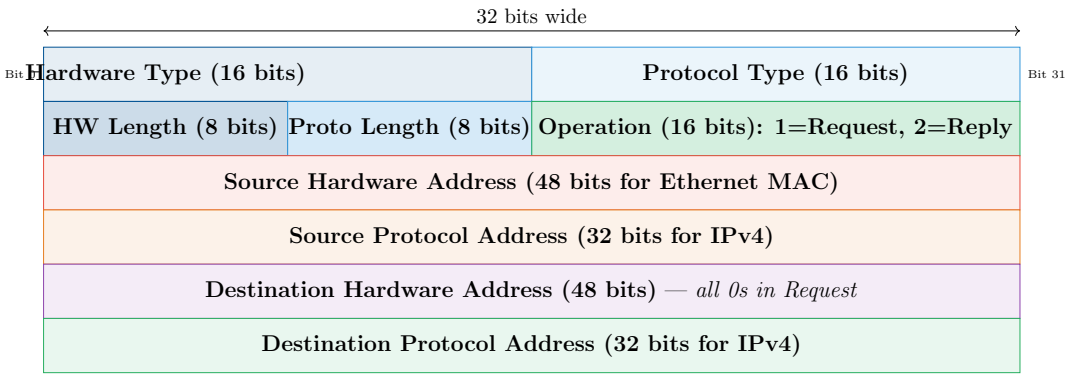
- It sends an **ARP reply** directly back (unicast) to L_A.
- The reply says: "I am MAC L_X, IP N_X."

3. **Caching:**
   - The asking node stores the mapping (IP → MAC) in its **ARP cache** (table) for future use.
   - Cache entries expire after a timeout (typically 20 minutes) to handle network changes.
   - This avoids broadcasting an ARP request every time a frame needs to be sent.



A broadcasts; all receive but only C replies; A stores L_C in ARP cache

2.2.2 ARP Packet Format



Field	Description
Hardware Type	Type of link layer (1 = Ethernet)
Protocol Type	Type of network layer (0x0800 = IPv4)
Hardware Length	MAC address length in bytes (6 for Ethernet)
Protocol Length	IP address length in bytes (4 for IPv4)
Operation	1 = ARP Request, 2 = ARP Reply
Source Hardware Addr	Sender's MAC address
Source Protocol Addr	Sender's IP address
Dest. Hardware Addr	Target MAC (all 0s in Request because unknown!)
Dest. Protocol Addr	Target IP address

Example 9.3: ARP Request Packet

Scenario: Host A (IP: 192.168.1.1, MAC: AA:BB:CC:11:22:33) wants to find the MAC address of Host B (IP: 192.168.1.5).

ARP Request packet contents:

- Hardware Type: 1 (Ethernet)
- Protocol Type: 0x0800 (IPv4)
- Operation: 1 (Request)
- Source Hardware: `AA:BB:CC:11:22:33` (A's MAC)
- Source Protocol: `192.168.1.1` (A's IP)
- Destination Hardware: `00:00:00:00:00:00` (UNKNOWN - that's what we're asking!)
- Destination Protocol: `192.168.1.5` (B's IP - this is what we know)

This packet is placed in a frame with destination MAC = `FF:FF:FF:FF:FF:FF` (broadcast).

Example 9.4: ARP Reply Packet

Scenario: Host B (IP: 192.168.1.5, MAC: DD:EE:FF:44:55:66) receives the ARP request and replies.

ARP Reply packet (from B to A):

- Operation: 2 (Reply)
- Source Hardware: `DD:EE:FF:44:55:66` (B's MAC — this is the answer!)
- Source Protocol: `192.168.1.5` (B's IP)
- Destination Hardware: `AA:BB:CC:11:22:33` (A's MAC)
- Destination Protocol: `192.168.1.1` (A's IP)

This reply is a unicast frame (sent only to A's MAC). Host A receives it and stores: `192.168.1.5` → `DD:EE:FF:44:55:66` in its ARP cache.

2.2.3 Complete Packet Journey: Alice to Bob (Full Example)

Let's walk through the **complete journey** of a packet from Alice to Bob, seeing how IP addresses stay the same but MAC addresses change at every hop.

Scenario Setup

- Alice's host: IP = N_A, MAC = L_A
- Router R1: Left interface IP = N_1, MAC = L_1; Right interface IP = N_2, MAC = L_2
- Router R2: Left interface IP = N_3, MAC = L_3; Right interface IP = N_4, MAC = L_4
- Bob's host: IP = N_B, MAC = L_B
- Links: Alice $\xrightarrow{\text{Link 1}}$ R1 $\xrightarrow{\text{Link 2}}$ R2 $\xrightarrow{\text{Link 3}}$ Bob

Step 1: Activities at Alice's site

1. Alice's **application layer** prepares a message for Bob.
2. It passes down to the **network layer**, which creates a datagram with:
   - Source IP = N_A, Destination IP = N_B (these never change!).
3. The network layer checks its **forwarding table**: to reach N_B, the next hop is Router R1 with IP N_1.
4. Alice runs **ARP**: "What is the MAC of N_1?" $\Rightarrow$ gets back L_1.
5. The network layer passes the datagram + destination MAC L_1 to the data-link layer.
6. The data-link layer creates **Frame on Link 1**:
$$\text{Dst MAC: } L_1 \mid \text{Src MAC: } L_A \mid \text{IP Datagram}(N_A \to N_B)$$
7. The physical layer converts to signals and sends on Link 1.

Step 2: Activities at Router R1

1. Physical layer receives signals, converts to bits, creates the frame.
2. **Data-link layer** decapsulates the frame (sees Dst MAC = L_1 = my MAC ✓), extracts the datagram.
3. Passes datagram up to the **network layer**.
4. Network layer reads: Destination IP = N_B. Consults forwarding table: next hop is R2 with IP N_3.
5. Runs **ARP**: "What is MAC of N_3?" $\Rightarrow$ gets L_3.
6. Passes datagram + destination L_3 **down to data-link layer of the right interface**.
7. Data-link creates **Frame on Link 2**:
$$\text{Dst MAC: } L_3 \mid \text{Src MAC: } L_2 \mid \text{IP Datagram}(N_A \to N_B)$$
8. Physical layer sends on Link 2.

Step 3: Activities at Router R2

Almost identical to R1:

1. Receives frame on Link 2. Decapsulates. Network layer reads destination = N_B.
2. Forwarding table says: N_B is directly connected (on Link 3). Next hop = N_B.
3. Runs **ARP**: "What is MAC of N_B?" $\Rightarrow$ gets L_B.
4. Creates **Frame on Link 3**:
$$\text{Dst MAC: } L_B \mid \text{Src MAC: } L_4 \mid \text{IP Datagram}(N_A \to N_B)$$
5. Sends on Link 3.

Step 4: Activities at Bob's site

1. Physical layer receives signals, converts to frame.
2. Data-link layer decapsulates (Dst MAC = L_B = my MAC ✓), extracts datagram.
3. Passes to network layer, which reads: Destination IP = N_B = *that's me!*
4. Passes the data up through transport layer to the application. Bob receives Alice's message!

Summary of Address Changes During Alice → Bob Journey

Frame on Link	Dst MAC (changes)	Src MAC (changes)	IP Src → Dst (FIXED)
Link 1 (Alice → R1)	L_1 (R1's left MAC)	L_A (Alice's MAC)	$N_A \to N_B$
Link 2 (R1 → R2)	L_3 (R2's left MAC)	L_2 (R1's right MAC)	$N_A \to N_B$
Link 3 (R2 → Bob)	L_B (Bob's MAC)	L_4 (R2's right MAC)	$N_A \to N_B$

IP addresses ($N_A \to N_B$): **NEVER CHANGE**. MAC addresses: **CHANGE AT EVERY HOP.**

Why don't we put all the intermediate router MACs in the original frame?
Because we don't know the path in advance! The Internet uses dynamic routing — the path can change. Also, each link uses a different protocol. The frame is rebuilt at every router with fresh MAC addresses for the next hop only.

2.2.4 ARP Caching

ARP Cache: Remembering Recent Answers

Running an ARP request for every single frame would create enormous broadcast traffic. **ARP caching** solves this:

- After each successful ARP exchange, the mapping (IP → MAC) is stored in an **ARP cache** (also called ARP table).
- Future frames to the same IP use the cached MAC immediately — no broadcast needed!
- Entries **expire** after a timeout (e.g., 20 minutes) to handle changes (NIC replaced, IP reassigned).
- When a node receives any ARP packet (request or reply), it caches the *sender's* mapping even without being the target (opportunity caching).

You can view your ARP cache: `arp -a` on Windows/Linux!

Chapter Summary

Key Takeaways — Chapter 9

Section 9.1 — Data-Link Layer Introduction:

- Communication at data-link layer is **node-to-node** (one hop at a time).
- **Nodes** = hosts + routers; **Links** = physical networks between nodes.
- Four services: **Framing**, **Flow Control**, **Error Control**, **Congestion Control**.
- Datagram is **re-encapsulated** into a new frame at each router (different frame format per link).
- Two link types: **Point-to-point** (dedicated) and **Broadcast** (shared).
- Two sublayers: **DLC** (common to all) and **MAC** (for broadcast links only).

Section 9.2.1 — Link-Layer Addressing:

- Link-layer addresses (MAC/physical addresses) are **48 bits** (6 bytes) in Ethernet.
- **Unicast**: LSB of first byte = 0. **Multicast**: LSB = 1. **Broadcast**: all bits = 1 (`FF:FF:FF:FF:FF:FF`).
- IP addresses NEVER change end-to-end. MAC addresses CHANGE at every hop.

Section 9.2.2 — ARP:

- ARP maps IP address → MAC address for the next hop.
- ARP Request = **broadcast** (all devices receive it). ARP Reply = **unicast** (only the requester).
- ARP packet: Hardware type, Protocol type, HW/Proto lengths, Operation, src/dst HW+proto addresses.
- ARP cache stores mappings to avoid repeated broadcasts. Expires after timeout.
- At each router: decapsulate frame, network layer looks up next hop IP, ARP finds next hop MAC, create new frame with new MACs.